TRMS Control: Zero-to-Master Technical Guide

N-MPC vs PID reference tracking on a Twin Rotor MIMO System · Oct 13, 2026 · @Aybars

Overview

On a 60 s simulated test, the Nonlinear Model Predictive Controller (N-MPC) cut the integrated tracking error by a factor of about 5.4 in pitch and 4.4 in yaw compared with an auto-tuned PID. It settled all 6 reference steps; the PID settled 1.

This guide explains that result from first principles. It assumes you know calculus, linear algebra and basic Newtonian mechanics, but no control theory. By the end you should be able to derive the plant equations, write both control laws, follow every line of the code, and read every number and graph the program produces.

What the study does

The project simulates a Twin Rotor MIMO System (TRMS), a laboratory helicopter model, using a 6-state nonlinear model of the Feedback Instruments 33-220. Two controllers try to make the beam follow the same sequence of step changes in pitch and yaw angle:

1. a decentralised **PID**: one PID loop drives pitch through the main rotor, a second drives yaw through the tail rotor;
2. an **N-MPC**: at every sample it solves an optimisation over the next 1.9 s using the full nonlinear model.

Both controllers are tuned automatically by the same optimiser against the same cost function, then run on a separate 60 s evaluation scenario. The program logs every state and controller variable and computes standard performance metrics.

How this guide is organised

Part	Question it answers
1. Theoretical foundations	What are the physics, and how do PID, N-MPC and auto-tuning work mathematically?

Part	Question it answers
2. Codebase architecture	Which file does what, how data flows, how CasADi, IPOPT and SciPy are used, how caching works
3. Data dictionary and metrics	What each output file contains; the exact definition of every metric
4. Visual analytics	How to read each generated plot
5. Results, limitations, glossary	What the numbers mean and what they do not prove

Three scope notes before you start

These correct common assumptions about this kind of project; the rest of the guide is consistent with them.

- **Libraries.** The N-MPC is built directly with CasADi and solved by IPOPT. The do-mpc library is not used. SciPy provides the auto-tuning optimiser.
- **Constraints.** The N-MPC enforces hard limits on the two inputs only. State constraints are explained in the theory, but none are imposed in this code.
- **Foresight.** The N-MPC predicts how the beam will move in response to its own planned inputs. It is not told about future reference steps (`reference_preview: false`), so it reacts to a step only once the step happens, exactly like the PID.

Part 1.1 — The TRMS: physics and nonlinear model

The TRMS is a 2-input, 2-output system whose six internal states are linked by trigonometric, polynomial and gyroscopic terms. Those terms make it nonlinear and cross-coupled: each input moves both outputs.

What a Twin Rotor MIMO System is

A TRMS is a beam pivoted at its centre on a fixed base. The pivot lets the beam rotate about two axes:

- **pitch**, alpha_v: rotation in the vertical plane (nose up or down);
- **yaw**, alpha_h: rotation in the horizontal plane (turning left or right).

A **main rotor** at one end of the beam pushes mostly vertically and controls pitch. A **tail rotor** at the other end pushes mostly horizontally and controls yaw. A counterweight on a short arm sets the beam's resting attitude. Each rotor is driven by a DC motor whose voltage is the control input. The inputs are normalised to the range −1 to +1 and called u_v (main) and u_h (tail).

"MIMO" means *multiple-input multiple-output*. Here there are 2 inputs and 2 measured outputs, and neither input acts on only one output. Laboratories use the TRMS because it reproduces, at bench scale, the problems of a real helicopter: unstable-looking lightly damped motion, actuator lag, saturation and strong axis coupling.

The state vector

The model uses six states and two inputs.

Symbol	Code name	Meaning	Unit
x1 = alpha_v	`pitch`	pitch angle	rad
x2 = S_v	`S_v`	pitch momentum state (defined below)	rad/s
x3 = u_vv	`u_vv`	main motor internal (lagged) voltage	–
x4 = alpha_h	`yaw`	yaw angle	rad
x5 = S_h	`S_h`	yaw momentum state	rad/s
x6 = u_hh	`u_hh`	tail motor internal (lagged) voltage	–
u1 = u_v	`u_pitch`	main motor command	–
u2 = u_h	`u_yaw`	tail motor command	–

The nonlinear state equations

The full model, in the form dx/dt = f(x, u), is:

```
\begin{aligned}
\dot{\alpha}_v &= \Omega_v = S_v + \frac{J_{tr}\,\omega_t(u_{hh})}{J_v} \\
\dot{S}_v &= \frac{1}{J_v}\Big[\, l_m S_f F_v\big(\omega_m(u_{vv})\big) +
g\big((A-B)\cos\alpha_v - C\sin\alpha_v\big) - \Omega_h^2 H
\sin\alpha_v\cos\alpha_v - K_v\,\Omega_v \Big] \\
\dot{u}_{vv} &= \frac{-u_{vv} + K_{mr}\,u_v}{T_{mr}} \\
\dot{\alpha}_h &= \Omega_h = S_h +
\frac{J_{mr}\,\omega_m(u_{vv})\cos\alpha_v}{J_h(\alpha_v)} \\
\dot{S}_h &= \frac{1}{J_h(\alpha_v)}\Big[\, l_t S_f
F_h\big(\omega_t(u_{hh})\big)\cos\alpha_v - K_h\,\Omega_h \Big] \\
\dot{u}_{hh} &= \frac{-u_{hh} + K_{tr}\,u_h}{T_{tr}} \\[4pt]
J_h(\alpha_v) &= D\cos^2\alpha_v + E\sin^2\alpha_v + F
\end{aligned}
```

Omega_v and Omega_h are the true angular velocities of the beam. The code logs them as `pitch_rate` and `yaw_rate`.

Reading the equations term by term

Newton's second law for rotation. Each momentum equation has the form "inertia × angular acceleration = sum of torques". The pitch torque sum has four parts, and the code logs each one separately:

```
J_v\,\dot{S}_v = \underbrace{l_m S_f F_v}_{M_{aero,v}} \;+\;
\underbrace{g\big((A-B)\cos\alpha_v - C\sin\alpha_v\big)}_{M_{grav}}
\;\underbrace{-\;\Omega_h^2 H\sin\alpha_v\cos\alpha_v}_{M_{cent}}
\;\underbrace{-\;K_v\Omega_v}_{M_{fric,v}}
```

- **Aerodynamic torque, M_aero,v** (`torque_aero_pitch`). The main rotor's thrust S_f·F_v (in newtons) acts at lever arm l_m = 0.24 m.
- **Gravity torque, M_grav** (`torque_gravity_pitch`). The beam is not balanced. A, B and C are lumped mass moments of the tail side, the main-rotor side and the counterweight with beam. Because the arms rotate with the beam, the vertical weights produce torques proportional to cos alpha_v and sin alpha_v. This makes the gravity term behave like a nonlinear spring that pulls the beam toward its rest angle.
- **Centrifugal torque, M_cent** (`torque_centrifugal_pitch`). When the beam spins in yaw at rate Omega_h, its masses are flung outward. That outward pull produces a pitch torque proportional to Omega_h² and sin(2 alpha_v).
- **Viscous friction, M_fric,v** (`torque_friction_pitch`). Bearing and air drag oppose motion in proportion to angular velocity, with coefficient K_v.

The yaw equation has the same structure with two torques, logged as `torque_aero_yaw` and `torque_friction_yaw`. The tail thrust acts at arm l_t = 0.25 m. It is multiplied by cos alpha_v because only the horizontal component of a tilted beam's tail thrust turns it in yaw. The yaw inertia J_h depends on pitch, because a tilted beam has its masses closer to the vertical axis.

Actuators: motor lag and rotor characteristics

Each DC motor cannot change speed instantly. The model represents this as a first-order lag. The internal state u_vv (or u_hh) chases the command with a time constant T_mr = 1.432 s for the main motor and T_tr = 0.3842 s for the tail motor. After one time constant, the internal state has covered 63 % of a step in command. The main motor is therefore almost four times slower than the tail motor, which matters a great deal for pitch control.

The lagged voltage sets the rotor speed through a measured **static characteristic**, and the speed sets the thrust through a second one. Both are polynomials fitted to laboratory data, with coefficients ordered from the highest power:

```
\begin{aligned}
\omega_m(u) &= 90.90u^6 + 599.73u^5 - 129.26u^4 - 1238.64u^3 + 63.45u^2 +
1238.41u \quad [\text{rad/s}] \\
F_v(\omega) &= -3.48\!\times\!10^{-12}\omega^5 +
1.09\!\times\!10^{-9}\omega^4 + 4.123\!\times\!10^{-6}\omega^3 -
1.632\!\times\!10^{-4}\omega^2 + 9.544\!\times\!10^{-2}\omega \\
\omega_t(u) &= 2020u^5 + 194.69u^4 - 4283.15u^3 - 262.87u^2 + 3796.83u \quad
[\text{rad/s}] \\
F_h(\omega) &= -3\!\times\!10^{-14}\omega^5 + 1.595\!\times\!10^{-11}\omega^4
+ 2.511\!\times\!10^{-7}\omega^3 - 1.808\!\times\!10^{-4}\omega^2 +
0.0801\,\omega
\end{aligned}
```

The scaling factor S_f converts F into newtons. At full main input (u = 1) the main rotor turns at about 625 rad/s (about 6,000 rpm) and produces about 0.70 N of thrust. The input-to-thrust relation is strongly curved, so the same change in u produces a different change in thrust at different operating points. This is the main source of the plant's nonlinearity.

Cross-coupling: why each input moves both axes

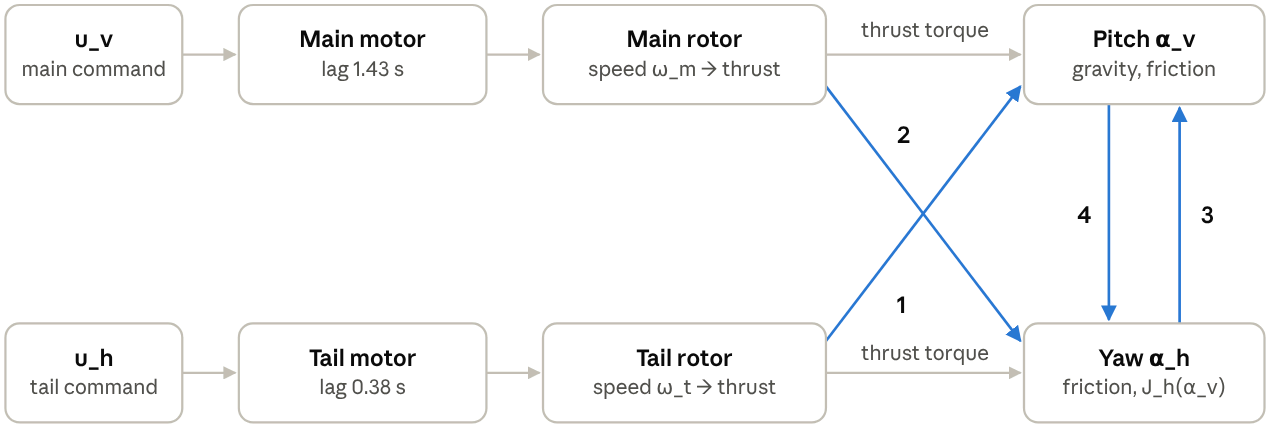
Cross-coupling means one input affects both outputs. The model contains four coupling paths, drawn in the diagram below.

1. **Tail rotor → pitch (reaction momentum).** The term J_tr·omega_t/J_v appears directly in the pitch rate. S_v stores the beam's angular momentum and cannot jump. So when the tail rotor speeds up, conservation of angular momentum makes the beam's pitch rate change at once. This is Newton's third law: spinning a rotor one way pushes the body the other way. In the results, every yaw step knocks the N-MPC's pitch by up to 17°.
2. **Main rotor → yaw (reaction momentum).** Likewise, J_mr·omega_m·cos(alpha_v)/J_h enters the yaw rate. Changing the main rotor speed turns the beam in yaw.
3. **Yaw → pitch (centrifugal).** Yaw rotation creates the pitch torque −Omega_h²·H·sin(alpha_v)·cos(alpha_v).
4. **Pitch → yaw (geometry).** The pitch angle scales the useful tail thrust by cos alpha_v and changes the yaw inertia J_h(alpha_v).

A controller that treats pitch and yaw as two independent problems ignores all four paths. It therefore sees their effects only as unexplained disturbances, after they have already happened.

Each input reaches both axes through four coupling paths

Grey: intended signal paths. Blue: cross-coupling paths, numbered as in the list above.



1 = tail-rotor reaction momentum → pitch

2 = main-rotor reaction momentum → yaw

3 = centrifugal torque, yaw rate → pitch

4 = geometry: cos α_v and J_h(α_v) → yaw

TRMS signal and coupling paths · from the model equations in trms/model.py

A decentralised controller only "sees" the grey paths; the blue ones reach it later, as unexplained errors.

Physical parameters

All values come from Table I of arXiv:1306.4883. The only change is the linear coefficient of F_h: the paper prints 0.8080, which would make the tail rotor about four times stronger than the main rotor, so the code uses 0.0801.

Symbol	Value	Unit	Meaning
A	0.0946875	kg·m	tail-side gravity moment term
B	0.11046	kg·m	main-side gravity moment term
C	0.01986	kg·m	counterweight and beam gravity term
D, E, F	0.04988, 0.004745, 0.006230	kg·m²	yaw inertia coefficients
H	0.048210	kg·m²	centrifugal coefficient
J_v	0.055448	kg·m²	pitch inertia
J_mr	1.6543×10^{-5}	kg·m²	main rotor and motor inertia
J_tr	2.65×10^{-5}	kg·m²	tail rotor and motor inertia
l_m, l_t	0.24, 0.25	m	main and tail rotor arms
S_f	8.43318×10^{-4}	–	thrust scaling
T_mr, T_tr	1.432, 0.3842	s	motor time constants
K_mr, K_tr	1, 1	–	motor static gains
K_v, K_h	0.0095, 0.00545371	N·m·s/rad	viscous friction
g	9.81	m/s²	gravity

(The paper lists A–C in kg·m²; multiplied by g they give torques in N·m, which is how the code uses them.)

Equilibrium, trim and the reachable envelope

An **equilibrium** is a state where nothing changes: every derivative is zero. With both rotors off, gravity alone balances when (A − B)·cos alpha_v = C·sin alpha_v:

$$\alpha_{v,\text{rest}} = \arctan\!\left(\frac{A-B}{C}\right) = \arctan\!\left(\frac{-0.01577}{0.01986}\right) = -0.671\ \text{rad} \; (-38.5^\circ)$$

Every simulation starts from this resting state. To hold any other pitch angle, the main rotor must supply exactly the torque gravity removes. The constant input that does this is the **trim** input, found by solving dS_v/dt = 0 at that angle. The code does this with `trim_main_input` using Brent's method.

Pitch held	Trim input u_v
−0.4 rad (−23°)	0.40
−0.2 rad (−11.5°)	0.59
0.0 rad (0°)	0.96
above about +0.09 rad	no solution with u_v ≤ 1

With this parameter set, the beam cannot be held above about +0.09 rad. This is why every reference in the study lies between −0.45 and −0.05 rad.

Two numbers that explain the whole comparison

Linearising the pitch dynamics around −0.2 rad gives a gravity "spring" stiffness of about 0.22 N·m/rad. With J_v = 0.0554 kg·m², this means:

$$\omega_n = \sqrt{\frac{0.22}{0.0554}} \approx 2.0\ \text{rad/s}, \qquad \zeta = \frac{K_v}{2\, J_v\, \omega_n} = \frac{0.0095}{2 \times 0.0554 \times 2.0} \approx 0.04$$

A damping ratio zeta of 0.04 is extremely low: left alone, the beam swings back and forth for many cycles. On top of that, the slope of thrust with respect to u_v is about 1.9 times larger near u_v = 0.4 (pitch −0.4 rad) than near u_v = 0.6 (pitch −0.2 rad). So the plant has very little natural damping and an effective gain that changes with operating point. A single fixed set of PID gains cannot be ideal everywhere, as Part 1.2 explains.

Part 1.2 — PID control

A PID controller computes the input from the present error, its accumulated past and its current trend. It needs no model of the plant, which is its great strength and, on this plant, its main weakness.

Feedback and the tracking error

In feedback control, the controller compares a **reference** r(t) (the desired angle) with the measured **output** y(t) (the actual angle). The difference is the tracking error:

```
e(t) = r(t) - y(t)
```

The controller's job is to drive e(t) to zero quickly, without large swings, and with inputs the actuators can deliver.

The continuous-time PID law

The textbook PID law is:

```
u(t) = K_p\, e(t) \;+\; K_i \int_0^{t} e(\tau)\, d\tau \;+\; K_d\,
\frac{de(t)}{dt}
```

In the Laplace domain (s is the complex frequency variable), the same controller is the transfer function:

```
C(s) = \frac{U(s)}{E(s)} = K_p + \frac{K_i}{s} + K_d\, s
```

Each term has a distinct physical role.

- **Proportional (P), Kp.** Pushes in proportion to the present error, like a spring. A larger Kp gives a faster response but, past a point, overshoot and oscillation.
- **Integral (I), Ki.** Accumulates past error. It keeps growing while any error remains, so it removes steady-state error. On the pitch axis this is essential: holding −0.2 rad needs a constant u_v of about 0.59, and only the integral can supply a constant output at zero error.
- **Derivative (D), Kd.** Reacts to how fast the error is changing, like a damper or shock absorber. It provides the damping the beam lacks (zeta ≈ 0.04).

What the code actually implements

The textbook law needs four practical changes before it works on real or realistically simulated hardware. `trms/controllers/pid.py` implements all four.

1. Derivative on measurement. At a step in the reference, de/dt is infinite for an instant, and a derivative of e would fire an enormous spike ("derivative kick"). Because r is constant between steps, de/dt = −dy/dt there, so the code differentiates the measurement instead:

```
D(t) = -K_d\, \frac{dy(t)}{dt}
```

2. Derivative filter. Pure differentiation amplifies high-frequency noise without limit. A first-order low-pass filter with time constant T_f = 0.1 s limits it:

```
D(s) = -\frac{K_d\, s}{T_f\, s + 1}\, Y(s)
```

3. Discretisation. A digital controller runs at fixed sample instants t_k = k·Ts, with Ts = 0.05 s. Applying the backward-Euler approximation (dD/dt ≈ (D_k − D_k−1)/Ts) to the filter, and a rectangle rule to the integral, gives the update equations used in `pid_step`:

```
\begin{aligned}
e_k &= r_k - y_k \\
P_k &= K_p\, e_k \\
I_k &= I_{k-1} + K_i\, T_s\, e_k \\
D_k &= \frac{T_f}{T_f + T_s}\, D_{k-1} \;-\; \frac{K_d}{T_f + T_s}\,\big(y_k
- y_{k-1}\big) \\
v_k &= P_k + I_k + D_k \\
u_k &= \operatorname{sat}(v_k) = \min\big(\max(v_k,\, -1),\, 1\big)
\end{aligned}
```

The code logs P_k, I_k and D_k as `pid_P_*`, `pid_I_*`, `pid_D_*`; the unsaturated sum v_k as `u_unsat_*`; and the applied u_k as `u_pitch` and `u_yaw`.

4. Anti-windup: conditional integration plus an integrator clamp. When the output is clamped at ±1, the plant receives less than the controller asks for. The integral would keep growing ("winding up") and later cause a long, large overshoot while it unwinds. The code prevents this in two layers.

Layer 1, conditional integration. Compute a tentative integral and output:

$$I_k^{\text{try}} = I_{k-1} + K_i\, T_s\, e_k, \qquad v_k^{\text{try}} = P_k + I_k^{\text{try}} + D_k$$

If the tentative output would cross a limit *and* the error pushes further in that direction, the integrator advances only as far as the value that puts the output exactly on the limit. It never goes beyond that value and never moves backwards:

$$I_k = \begin{cases} \max\!\big(I_{k-1},\; \min(I_k^{\text{try}},\; u_{\max} - P_k - D_k)\big) & \text{if } v_k^{\text{try}} > u_{\max} \wedge e_k > 0 \\ \min\!\big(I_{k-1},\; \max(I_k^{\text{try}},\; u_{\min} - P_k - D_k)\big) & \text{if } v_k^{\text{try}} < u_{\min} \wedge e_k < 0 \\ I_k^{\text{try}} & \text{otherwise} \end{cases}$$

Once the output already sits on the limit, the bound equals I_{k-1}, so the integrator is frozen, which is the classic conditional-integration rule. The first version of the code simply kept I_{k-1} whenever the tentative output crossed a limit. If a single integration step jumped past the limit, the integrator then never moved, and with a large Ki the output could stay stuck at 0 under a large error. The rule above was introduced in the 2026-10-14 code review to fix this.

Layer 2, integrator clamp. Layer 1 cannot stop I from growing while a large opposite P or D keeps the *sum* inside the limits. So the integral term alone is also clamped to the actuator range:

$$I_k \leftarrow \min\big(\max(I_k,\; u_{\min}),\; u_{\max}\big)$$

The flags `pid_integrator_frozen_pitch` and `pid_integrator_frozen_yaw` record the samples where layer 1 limited the integrator (6 samples for pitch in the reference run).

Decentralised MIMO structure

The project uses two independent single-input single-output (SISO) loops. Pitch error drives the main rotor, and yaw error drives the tail rotor. In matrix form the controller is diagonal:

$$\begin{bmatrix} U_v(s) \\ U_h(s) \end{bmatrix} = \begin{bmatrix} C_v(s) & 0 \\ 0 & C_h(s) \end{bmatrix} \begin{bmatrix} E_v(s) \\ E_h(s) \end{bmatrix}$$

The zeros off the diagonal are the key point: neither loop knows the other exists. The auto-tuned gains are:

Loop	Kp	Ki	Kd
Pitch (u_v)	0.456	0.396	0.132
Yaw (u_h)	0.181	0.001	0.651

Ki for yaw sits at the lower search bound (10^{-3}), so the yaw loop is effectively a PD controller. That is physically sensible. Yaw has no gravity to fight, so it needs no constant input at rest and therefore no integral action.

Why a PID struggles on a coupled, nonlinear MIMO plant

The PID's difficulties here have four separate causes. The first two dominate the results of this study.

1. **Fixed gains on a plant whose gain changes.** The thrust slope with respect to u_v is about 1.9 times larger near pitch −0.4 rad than near −0.2 rad. Loop gain is the product of controller gain and plant gain. So gains that are just stable at −0.2 rad give almost twice the loop gain at −0.4 rad, which pushes the loop past its stability margin. This is exactly what the evaluation shows: a sustained oscillation at −0.2 rad that grows into instability after the step to −0.4 rad at t = 40 s.
2. **Very low damping behind a slow actuator.** The D term is the only source of damping. But it acts through the main motor, whose 1.43 s lag delays every correction. At the beam's natural frequency (about 2 rad/s), this lag costs roughly 70° of phase. The D term can therefore only partly cancel the swinging. More gain makes the swings worse; less gain makes the response slow. The optimiser's best compromise still oscillates.
3. **Coupling seen as disturbance.** Each loop sees the other axis only through its effects. When the yaw loop changes tail rotor speed, the reaction momentum tilts the pitch axis at once. The pitch loop corrects only after an error appears, and its correction changes main rotor speed, which in turn disturbs yaw. In general such loops can excite each other ("fight"). In this particular run the PID's yaw loop is slow and gentle, so the interaction is mild; the dominant failure is the pitch oscillation of causes 1 and 2.
4. **Limits without foresight.** The PID cannot plan around the ±1 input limit. Anti-windup stops the integral from making things worse, but the controller still does not know a limit is coming until it reaches it.

These are not tuning mistakes. A 4,000-point random search over the pitch gains found nothing better than the optimiser's result (best objective 20.8 against 19.5). The usual engineering remedies are gravity feed-forward (add the trim input to the PID output) and gain scheduling (change gains with pitch angle); both are deliberately left out so the comparison stays pure PID vs N-MPC.

Part 1.3 — Nonlinear Model Predictive Control

An N-MPC replaces a fixed control law with an optimisation solved again at every sample. It simulates the plant's nonlinear model over a future window, chooses the input sequence that minimises a cost, applies only the first input, and repeats 50 ms later.

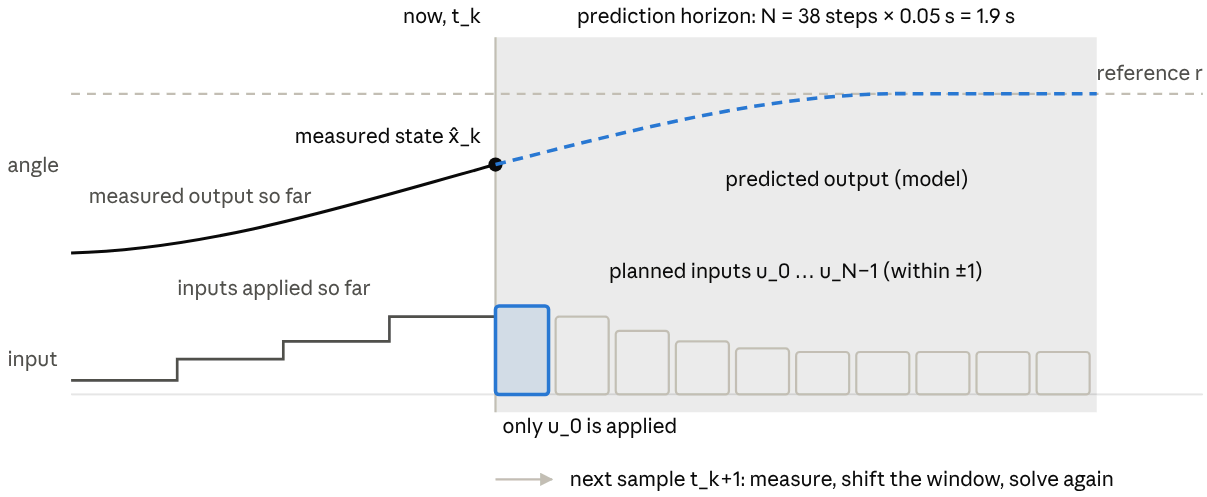
The receding-horizon principle

At each sample time t_k the controller performs four steps:

1. **Measure** the current state x̂_k of the plant.
2. **Predict and optimise.** Using the model, find the sequence of N future inputs u_0, …, u_N−1 that minimises the cost while respecting all constraints.
3. **Apply** only the first input u_0 to the plant for one sample period Ts.
4. **Shift** the window one step forward and go back to step 1.

The window always looks the same distance into the future, so it is called a **receding horizon**. Re-planning at every step is what turns an open-loop plan into feedback: model errors and disturbances show up in the next measurement, and the next plan corrects for them.

Plan 1.9 s ahead, apply only the first input, then re-plan



receding-horizon principle · schematic, not plotted data

The whole plan is recomputed 50 ms later from a fresh measurement, so only its first input ever reaches the motors.

The prediction model

The optimiser needs a discrete-time model x_k+1 = F(x_k, u_k). The code builds F by integrating the continuous model f from Part 1.1 with the classical fourth-order Runge–Kutta (RK4) method. With step h, one RK4 step is:

```
\begin{aligned}
k_1 &= f(x,\, u) \\
k_2 &= f\!\left(x + \tfrac{h}{2}k_1,\, u\right) \\
k_3 &= f\!\left(x + \tfrac{h}{2}k_2,\, u\right) \\
k_4 &= f(x + h\,k_3,\, u) \\
x^{+} &= x + \tfrac{h}{6}\,(k_1 + 2k_2 + 2k_3 + k_4)
\end{aligned}
```

The input is held constant over each sample (a "zero-order hold"). The prediction model takes 2 RK4 steps per sample (h = 0.025 s). The simulated plant takes 10 steps per sample (h = 0.005 s), so it plays the role of the "real" system. Both use exactly the same equation code, so in this study the prediction model is essentially perfect.

The horizon

The **prediction horizon** N is the number of future samples the controller plans over. The tuned value is N = 38, which with Ts = 0.05 s covers:

```
T_{\text{horizon}} = N\,T_s = 38 \times 0.05\ \text{s} = 1.9\ \text{s}
```

The horizon must be long enough to see the consequences of an action. The main motor alone needs 1.43 s to cover 63 % of a commanded change, and the beam swings with a period of about 3 s. With N = 10 (0.5 s), the controller cannot see that cutting thrust now leads to a fall later, and in testing the beam fell to the divergence limit. A longer horizon sees further but makes each optimisation larger and slower. The tuner therefore searched N between 15 and 40.

The cost function

The cost J scores a whole planned trajectory. In this project it penalises tracking error, angular rates and input *changes*:

```
J = \sum_{k=1}^{N} w_k \Big[\, q_p\, e_{p,k}^2 + q_y\, e_{y,k}^2 + q_{dp}\,
\Omega_{v,k}^2 + q_{dy}\, \Omega_{h,k}^2 \Big] \;+\; \sum_{k=0}^{N-1} \Big[\,
r_p\, \Delta u_{p,k}^2 + r_y\, \Delta u_{y,k}^2 \Big]
```

with tracking errors and input increments:

```
e_{p,k} = \alpha_{v,k} - r_p, \qquad e_{y,k} = \alpha_{h,k} - r_y, \qquad
\Delta u_k = u_k - u_{k-1}, \qquad w_k = \begin{cases} 5 & k = N \\ 1 & k < N
\end{cases}
```

Here u_−1 is the input actually applied at the previous sample, so the first increment penalises a jump relative to what the actuator is already doing. In compact matrix notation:

```
J = \sum_{k=1}^{N} w_k\, z_k^{\top} Q\, z_k + \sum_{k=0}^{N-1} \Delta
u_k^{\top} R\, \Delta u_k, \qquad z_k = \begin{bmatrix} e_{p,k} \\ e_{y,k} \\
\Omega_{v,k} \\ \Omega_{h,k} \end{bmatrix}
```

```
Q = \operatorname{diag}(1,\; 3.05,\; 0.0042,\; 0.343), \qquad R =
\operatorname{diag}(0.0128,\; 0.352)
```

Each weight expresses a priority.

- **Q on errors** (q_p = 1 fixed as the scale, q_y = 3.05). The tuner found yaw error worth about three times pitch error per rad².
- **Q on rates** (0.0042 and 0.343). These act like damping: the controller is discouraged from producing fast angular motion, which reduces overshoot. Yaw rate is penalised far more than pitch rate.
- **R on increments** (0.0128 and 0.352). These discourage jumpy inputs. The small pitch value means the controller is allowed to slam the main motor input; the larger yaw value keeps the tail input smoother.
- **Terminal weight** (w_N = 5). The last predicted state counts five times. This approximates the cost of everything after the horizon and pulls the plan toward ending near the reference.

Penalising Δu rather than u is deliberate. Holding pitch needs a large constant u_v (about 0.4–0.96), so a penalty on u itself would fight the very input needed to hold the beam up and leave a steady-state error. A penalty on Δu costs nothing once the input is steady.

Constraints

Constraints are hard limits the plan must obey. MPC's ability to include them directly is one of its main advantages over PID.

Input saturation (used here). The motor voltages are limited to $-1 \leqslant u \leqslant 1$. The PID clips its output *after* computing it, so its internal plan and reality disagree whenever it saturates. The N-MPC includes the limit *inside* the optimisation, so every plan it makes is one the actuators can actually execute. This is why its inputs often sit exactly at ±1 for short bursts: the optimum is frequently on the boundary.

State constraints (not used here). In general an MPC can also require predicted states to stay in a safe set, written $g(x_k) \leqslant 0$. For a TRMS this could be a pitch range, for example $-1 \leqslant \alpha_v \leqslant 0.1$. In practice state limits are usually made *soft* by adding a slack variable $s \geqslant 0$ with a heavy penalty, so the problem stays solvable when the limit cannot be met:

```
\alpha_{v,k} \le \alpha_{\max} + s_k, \qquad s_k \ge 0, \qquad J \leftarrow J + \rho \sum_k s_k^2
```

This project imposes no state constraints, only input bounds. The decision-variable bounds on states are set to ±infinity in `nmpc.py`.

The full optimisation problem

At every sample, the controller solves this nonlinear program (NLP):

```
\begin{aligned}
\min_{x_0,\dots,x_N,\; u_0,\dots,u_{N-1}} \quad & J(x, u) \\
\text{subject to} \quad & x_0 = \hat{x}(t_k) && \text{(start from the
measured state)} \\
& x_{k+1} = F(x_k,\, u_k), \quad k = 0,\dots,N-1 && \text{(nonlinear
dynamics)} \\
& u_{\min} \le u_k \le u_{\max}, \quad k = 0,\dots,N-1 && \text{(input
saturation)}
\end{aligned}
```

Multiple shooting: how the problem is set up

There are two ways to pose this NLP. In *single shooting* only the inputs are variables, and the states are computed by simulating forward. In **multiple shooting**, which this code uses, the predicted states are variables too, and the dynamics become equality constraints that the solver enforces. Multiple shooting is better conditioned for an unstable or lightly damped plant: a small change in an early input no longer swings the whole predicted trajectory wildly.

For N = 38 the problem has:

Quantity	Formula	Count
State variables	6 (N + 1)	234
Input variables	2 N	76
Total decision variables w	6(N+1) + 2N	310
Equality constraints (start + dynamics)	6 (N + 1)	234
Bounds on inputs	2 N	76
Parameters p (state, references, previous input)	6 + 2N + 2	84

How IPOPT solves it

IPOPT (Interior Point OPTimizer) is a solver for large nonlinear programs. It handles the bounds by adding a logarithmic barrier term, which grows to infinity at the limits and keeps iterates strictly inside them:

$$\min_{w}\; J(w) - \mu \sum_{i} \Big[\ln\big(w_i - w_i^{\min}\big) + \ln\big(w_i^{\max} - w_i\big) \Big] \quad \text{s.t. } g(w) = 0$$

It solves the optimality (Karush–Kuhn–Tucker, KKT) conditions of this problem with Newton's method, then shrinks the barrier parameter mu toward zero so the solution approaches the true constrained optimum. The KKT conditions use the Lagrangian:

$$\mathcal{L}(w, \lambda) = J(w) + \lambda^{\top} g(w), \qquad \nabla_w \mathcal{L} = 0,\;\; g(w) = 0$$

Newton's method needs first and second derivatives of J and g. CasADi computes them exactly by **automatic differentiation**, not finite differences, which makes each iteration accurate and fast.

Warm starting. Successive problems differ only by one sample, so the previous sample's answer is an excellent first guess. The code shifts the previous solution one step forward: each predicted state and input moves one stage earlier, the last stage is repeated, and the first state is replaced by the new measurement. The Lagrange multipliers of the dynamics constraints and input bounds are shifted the same way, so the primal and dual guesses describe the same time steps. IPOPT then starts with a small barrier parameter (mu_init = 10^{-3}) and an adaptive mu strategy. Warm starting roughly halved the closed-loop run time compared with a cold start.

The first version shifted the primal guess but passed the multipliers unshifted. Making the shift consistent (2026-10-14 review) cut the IPOPT iterations on the 60 s evaluation run from 2,075 to 1,891 (9 %). The closed-loop trajectory changed by at most 1.1×10^{-4} rad, which is solver-tolerance level.

Solver reuse. The weights q and r enter the NLP as *parameters*, appended to the parameter vector p, not as numeric constants. The expression graph, its exact derivatives and the IPOPT instance therefore depend only on the model, Ts, N, the terminal weight and the solver options. `_build_solver` builds them once per combination and caches them, so the tuner, which tries hundreds of weight sets, rebuilds a solver only when the horizon N changes. Before, every candidate paid about 1–3 s of construction.

Why N-MPC copes where PID struggles

- **It uses the nonlinear model.** The varying thrust slope, gravity curve and motor lag are all inside the prediction, so there is nothing for fixed gains to get wrong.

- **It knows the coupling.** The reaction-momentum and centrifugal terms are in the model, so planned tail inputs come with predicted pitch consequences. It still accepts pitch bumps of up to 17° at yaw steps: with q_y = 3.05, a fast yaw turn is worth more to the cost than a calm pitch axis, and without reference preview it cannot start preparing before the step arrives.
- **It plans within the limits.** Saturation is a constraint, not a surprise.
- **It plans ahead in time.** It sees that cutting thrust now causes a fall later and acts on that, which a PID, reacting only to present error, cannot.

Caveats

- **Full state.** The controller is given all six states exactly. On hardware only the two angles are measured, so an observer (for example an extended Kalman filter or moving-horizon estimator) would have to estimate S_v, S_h and the motor states.
- **Perfect model.** The prediction and the plant share equations. Real model mismatch will degrade the N-MPC more than the model-free PID.
- **No formal stability proof.** The terminal weight is a heuristic. A rigorous design would add a terminal constraint set and terminal cost with proven recursive feasibility.
- **Computation.** IPOPT averaged about 11–16 ms per 50 ms sample, but its slowest sample took between 110 and 310 ms in different runs on the same machine, and about 1–1.5 % of samples exceeded the budget. A real-time implementation would cap iterations or use a real-time iteration (RTI) scheme, such as those in acados.

Part 1.4 — Automatic tuning

Both controllers were tuned by the same global optimiser, differential evolution, minimising the same cost on the same tuning scenario. The PID reached J = 19.51 after 15,488 simulated runs; the N-MPC reached J = 2.36 after 224.

Why tune automatically, and fairly

A comparison is only meaningful if each controller is as good as it can reasonably be made. Hand-tuning one controller more carefully than the other would decide the outcome in advance. The project therefore uses one objective, one optimiser family and one tuning scenario for both. It then scores them on a *different* evaluation scenario, so neither is judged on the exact trajectory it was fitted to.

Scenario	Duration	Pitch steps (rad)	Yaw steps (rad)
Tuning	38 s	−0.30 at 0 s, −0.10 at 12 s, −0.45 at 22 s	+0.40 at 6 s, −0.30 at 18 s
Evaluation	60 s	−0.20 at 0 s, −0.05 at 20 s, −0.40 at 40 s	+0.50 at 10 s, −0.50 at 30 s, 0.00 at 50 s

Both scenarios start from the resting state (pitch −0.671 rad). The tuning scenario covers the same pitch range as the evaluation and ends with a 16 s hold, which exposes slow drift and growing oscillation.

The tuning objective

The cost function J, implemented in `metrics.tuning_objective`, is:

```
J = \underbrace{\sum_{k} \tau_k\, |e_{p,k}|\, T_s}_{\text{ITAE}_\text{pitch}}
+ \underbrace{\sum_{k} \tau_k\, |e_{y,k}|\, T_s}_{\text{ITAE}_\text{yaw}} +
0.02 \sum_{k} \big( |\Delta u_{p,k}| + |\Delta u_{y,k}| \big) + 0.05
\sum_{\text{steps } j} \max\!\big(0,\, \text{OS}_j - 5\big) + P_{\text{div}}
```

where tau_k is the time since the most recent reference change on either axis:

```
\tau_k = t_k - \max\{\, t_j^{\text{step}} \le t_k \,\}
```

and the divergence penalty is:

```
P_{\text{div}} = \begin{cases} 1000 & \text{if } \max_k |\alpha_{v,k}| > 1.4\
\text{rad} \;\vee\; \max_k |e_{y,k}| > 1.5\ \text{rad} \;\vee\; \text{the run
produced NaN or was flagged diverged} \\ 0 & \text{otherwise} \end{cases}
```

The terms reward four things: small error (ITAE), smooth inputs (total variation of u), limited overshoot (anything above 5 % is penalised), and stability (P_div).

Why ITAE instead of IAE

Plain IAE weighs an error equally whenever it occurs. ITAE multiplies each error by how long ago the last step happened, so an error that persists costs more and more. A fast transient that dies out is cheap; a slow drift or a sustained oscillation is expensive.

The project learned this the hard way. A first version used plain IAE and checked divergence only on pitch. The optimiser then chose near-zero yaw gains (Kp ≈ 0.01). Their slow drift stayed hidden inside the short tuning window but ran away to about 2,000° of yaw in the longer evaluation run. Switching to ITAE, adding the yaw divergence check and lengthening the final hold fixed this.

Differential evolution

Differential evolution (DE) is a population-based, derivative-free global optimiser. It is well suited here because J is noisy in shape: discontinuous where runs diverge, with many local minima, and with no gradient available. Each candidate is a parameter vector θ. The optimiser keeps a population of candidates and improves it generation by generation.

For each member x_i of generation g, SciPy's default `best1bin` strategy does three things.

1. Mutation. Build a mutant from the current best member and the difference of two random members r1 and r2:

```
v_i = x_{\text{best}} + F\,\big(x_{r_1} - x_{r_2}\big), \qquad F \sim \mathcal{U}(0.5,\, 1.0) \text{ (dithered each generation)}
```

2. Binomial crossover. Mix the mutant with the current member, coordinate by coordinate, with crossover probability CR = 0.7. One random coordinate j_rand is always taken from the mutant:

```
u_{i,j} = \begin{cases} v_{i,j} & \text{if } \rho_j < CR \;\vee\; j = j_{\text{rand}} \\ x_{i,j} & \text{otherwise} \end{cases}, \qquad \rho_j \sim \mathcal{U}(0,1)
```

3. Greedy selection. Keep whichever is better:

```
x_i^{(g+1)} = \begin{cases} u_i & \text{if } J(u_i) \le J(x_i) \\ x_i & \text{otherwise} \end{cases}
```

The initial population is a Sobol sequence, a low-discrepancy set that spreads points evenly over the search box. SciPy rounds a Sobol population up to a power of two, which is why the run counts below are 128 and 32 per generation. Updating is "deferred": a whole generation is evaluated before any member is replaced, which allows batch and parallel evaluation. The seed is fixed at 42, so tuning is reproducible.

Search spaces

Gains and weights are searched as base-10 logarithms. The right value of a gain may lie anywhere from 0.001 to 10, and a log scale gives each decade equal attention.

Controller	Dimension	Parameters searched (log10 bounds)
PID	6	Kp ∈ [−2, 1.5], Ki ∈ [−3, 1], Kd ∈ [−3, 1], each for pitch and yaw
N-MPC	6	q_y ∈ [−1, 1.5]; q_dp, q_dy, r_p, r_y ∈ [−4, 0]; integer N ∈ [15, 40] (linear)

For the N-MPC, q_p is fixed at 1. Multiplying every weight by the same constant leaves the optimal plan unchanged, so one weight must be fixed to remove that redundancy. The horizon N is an integer; SciPy's `integrality` option rounds that coordinate.

Tuning runs

	PID	N-MPC
Candidates per generation	128	32
Generations (incl. initial)	121	7
Simulated runs	15,488	224
Evaluation strategy	whole generation in one vectorised simulation (numpy PID, CasADi Function.map plant)	one closed-loop IPOPT run per candidate, in parallel processes, one cached solver per horizon
Wall time (2-core machine)	519 s	2,039 s
Final objective J	19.51	2.36

The two budgets differ by a factor of 70 because one N-MPC run costs far more than one PID run. Every N-MPC run solves about 760 nonlinear programs (one per sample), while the PID's whole population advances in a single array operation. Both searches stopped at

their iteration limit rather than at a convergence tolerance. More generations might still improve either result slightly.

The wall times above are from the original code, which produced the cached parameters. After the 2026-10-14 optimisation (Part 2, *Numerical safeguards and performance*), the same searches with the same settings and seeds take 142 s for the PID and 1,181 s for the N-MPC. They reach J = 19.51 and 2.36, the same optima within 0.3 %. The cached parameters were kept, and they score exactly as before under the new code (19.507 and 2.363).

DE is a heuristic and gives no guarantee of a global optimum. Here the PID result was checked independently: 4,000 random pitch-gain sets all scored worse.

Part 2 — Codebase architecture and execution

The repository is a small Python package, `trms/`, driven by one entry script, `main.py`. All physics lives in one function, `model.terms`, which every other part reuses: the simulated plant, the N-MPC's prediction model and the trim calculation.

Repository layout

```
TRMS-Kontrol/
├── main.py                      entry point: tune → simulate → log → plot → compare
├── requirements.txt             numpy, scipy, pandas, matplotlib, openpyxl, pyyaml, casadi
├── config/
│   ├── settings.yaml            every experiment setting
│   └── tuned_parameters.json    cache of tuned gains and weights + tuning reports
├── trms/
│   ├── params.py                physical parameters (validated on creation)
│   ├── model.py                 nonlinear dynamics, compiled plant, RK4, trim, envelope
│   ├── scenario.py              reference signals and step segmentation
│   ├── controllers/
│   │   ├── pid.py               discrete PID with two-layer anti-windup
│   │   └── nmpc.py              CasADi/IPOPT N-MPC, cached solver per horizon
│   ├── tuning.py                differential-evolution tuners
│   ├── cache.py                 signature-checked JSON cache (atomic writes)
```

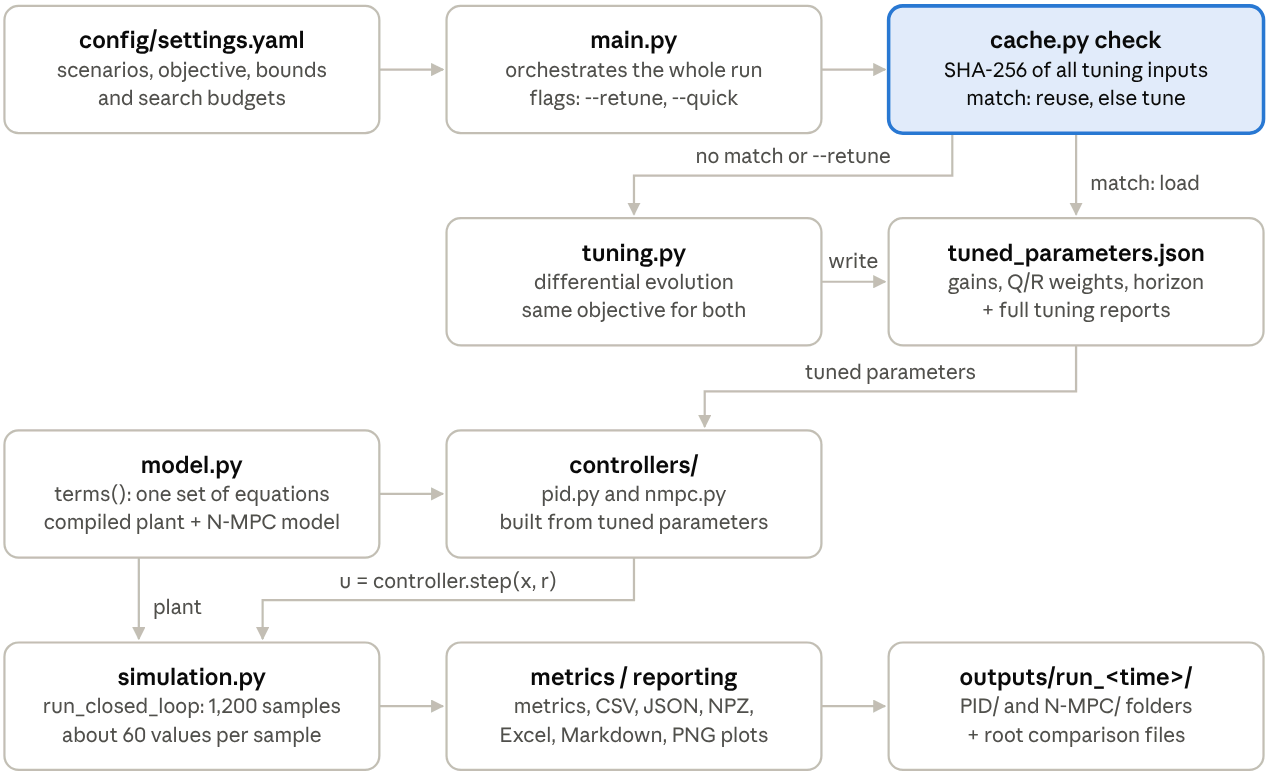
```
|   ├── simulation.py            closed-loop simulation, pre-allocated
telemetry
|   ├── metrics.py               metrics and the tuning objective
|   ├── plotting.py              figures
|   └── reporting.py             CSV / JSON / NPZ / Excel / Markdown writers
├── tests/test_core.py           regression tests (python tests/test_core.py)
└── docs/results/                figures and summary of the reference run
```

How one run executes

`python main.py` performs these steps in order:

1. **Parse arguments.** `--retune {pid,nmpc,all}` forces re-tuning; `--quick` shrinks the search budget and uses a separate cache file; `--settings`, `--cache` and `--output-root` override paths.
2. **Load settings** from `config/settings.yaml` and create the parameter set, the resting initial state and both scenarios.
3. **For each controller, check the cache.** Compute the signature of everything that affects tuning. If a cached entry has the same signature and no re-tune was forced, reuse it. Otherwise run the tuner and store the result.
4. **Build the two controllers** from the tuned parameters.
5. **Simulate** each controller on the 60 s evaluation scenario with `run_closed_loop`, which returns a telemetry table.
6. **Score** each run: per-step and whole-run metrics.
7. **Write each controller's folder** (`PID/`, `N-MPC/`): telemetry, metrics, parameters, figures.
8. **Write the comparison files** at the run's root: merged time series, metric tables, an Excel workbook, overlay plots and `summary.md`.
9. **Print** the summary table and the output path.

Settings flow through the cache check to simulation and the output folders



main.py data flow · from the repository code

The cache check is the only branch: a matching signature skips tuning entirely. The tuner also runs the model and the simulation, internally, to score each candidate.

File-by-file breakdown

`main.py` — orchestration only; it contains no physics or mathematics. It wires the modules together in the order above and timestamps each run folder (`outputs/run_YYYYmmdd-HHMMSS/`), so runs never overwrite each other.

`config/settings.yaml` — the single place to change the experiment. Its sections:

Section	Controls
`simulation`	sample time Ts = 0.05 s, plant RK4 sub-steps (10)
`scenarios`	the evaluation and tuning step sequences, as [axis, time, value]

Section	Controls
`objective`	weights and limits of the shared tuning cost
`pid`	derivative filter T_f, log10 search bounds, DE budget
`nmpc`	fixed q_p, terminal weight, prediction sub-steps, reference preview, IPOPT options, search bounds, DE budget
`quick`	reduced DE budgets for `--quick`
`output`	output root folder

`trms/params.py` — `TRMSParams`, a frozen (immutable) dataclass holding every physical constant and polynomial. Being immutable means no code can change a parameter by accident mid-run, and it makes the object hashable, so it can key the solver and integrator caches. On creation it validates itself. Every divisor in the model must stay positive: J_v, T_mr, T_tr and J_h(alpha_v) = D cos² + E sin² + F, which requires F + min(D, E) > 0. Friction and inertias must be non-negative and u_min < u_max. An invalid set raises a clear error instead of producing a division by zero deep inside a simulation. `to_dict()` serialises it for the cache signature, and `rest_pitch` computes the resting angle with `atan2(A - B, C)`, which cannot divide by zero and picks the stable root for either sign of C. It also defines the state names used as log column names.

`trms/model.py` — the physics.

- `polyval` evaluates a polynomial by Horner's rule. It works on plain floats, numpy arrays and CasADi symbols alike.
- `terms(x, u, p, sin, cos)` is the heart of the project. It returns the six derivatives and a dictionary of every intermediate quantity (rotor speeds, thrusts, each torque, J_h). The trigonometric functions are passed in as arguments. Called with `np.sin` it computes numbers; called with `ca.sin` it builds a symbolic expression graph for CasADi. One function therefore defines both the plant and the controller's model, so they cannot drift apart.
- `f_np` and `aux_np` are the numpy wrappers. They accept states of shape (6, T), so the simulation evaluates them once over a whole trajectory to build the telemetry.
- `rk4_np` integrates with RK4 under a zero-order-hold input. It is the numpy reference implementation that the tests compare against.

- `rk4_casadi` builds two CasADi `Function` objects: `F(x, u)`, the RK4 step over one sample, and `rates(x)`, which returns Omega_v and Omega_h for the cost. It is cached with `functools.lru_cache`, so identical requests reuse the already-built graph.
- `Plant` is the simulated plant: `rk4_casadi` with 10 sub-steps, evaluated by CasADi's SX virtual machine. One sample costs about 40 µs instead of about 3 ms for the numpy version, and the result is bit-identical. For a batch of shape (6, B) it uses `Function.map`, one compiled loop over the columns.
- `rest_state`, `trim_main_input` (SciPy `brentq` root-finding) and `pitch_envelope` compute the equilibria of Part 1.1.

`trms/scenario.py` — `Scenario` turns the YAML step list into a reference function r(t). `steps()` cuts the timeline into one segment per reference change. Each segment ends at the next change on *either* axis, so a step's metrics are not polluted by the coupling disturbance from a later step on the other axis.

`trms/controllers/pid.py` — `pid_step` implements the discrete PID of Part 1.2, including both anti-windup layers, on arrays of shape (2, B): two loops for B gain sets at once. `PIDController` wraps it with B = 1 for the logged simulation, so the tuner and the final run use identical arithmetic. `check_timing` rejects Ts ⩽ 0 and T_f < 0, the only cases where the divisor T_f + Ts could vanish, and non-finite gains are rejected at construction.

`trms/controllers/nmpc.py` — `_build_solver` builds the multiple-shooting NLP of Part 1.3. It creates CasADi `SX` symbols for X (6 × (N+1)) and U (2 × N), chains the dynamics constraints with `F`, accumulates the cost, and compiles everything into an IPOPT solver with `ca.nlpsol`. The parameter vector carries the current state, the references, the previous input and the six weights, so the result depends only on (model, Ts, N, terminal weight, sub-steps, options) and is cached. `NMPCController` fetches the cached solver at construction. Each `step()` call then only fills in numbers: the parameter vector, the shifted primal guess and the shifted multipliers. It calls the solver, applies the first input, and writes the whole predicted trajectory into pre-allocated arrays for the prediction plot. If a solve ever returns non-finite numbers, the controller follows its previous plan one more step and shifts it, so repeated failures keep advancing along that plan.

`trms/simulation.py` — `run_closed_loop` is the virtual laboratory. Before the loop it evaluates the whole reference signal at once and pre-allocates every buffer: states (T × 6), inputs (T × 2) and one typed column per controller log entry. Inside the loop it does only what must be sequential: ask the controller for an input, store the state and input, and advance the `Plant` by one sample. It stops early if the state becomes non-finite. After the loop, `_telemetry_frame` computes every derived quantity in one vectorised

pass over the whole (6, T) trajectory: derivatives, rotor speeds, torques, saturation flags, and running integrals via `cumsum`. The column names, order and dtypes are the same as before. `simulate_pid_batch` is the fast tuning path. It advances an entire PID population with one `Function.map` plant call per sample and keeps an explicit `diverged` mask: a member whose state becomes non-finite is flagged and frozen, so NaN never spreads into the objective.

`trms/metrics.py` — `step_metrics` (one reference step), `global_metrics` (the whole run) and `tuning_objective` (the cost J of Part 1.4, vectorised over a population). Part 3 defines each metric. A step window with fewer than two finite samples, such as one after a run stopped early, yields NaN metrics instead of crashing. In the objective, a non-finite score or a candidate flagged as diverged always receives the divergence penalty.

`trms/tuning.py` — `tune_pid` and `tune_nmpc` call `scipy.optimize.differential_evolution`. For the N-MPC, each candidate must run in a separate process, so the objective `_nmpc_objective` is a top-level function and receives only plain picklable data (`ctx`). Each worker process keeps its own solver cache, building a solver the first time it meets a horizon N and reusing it for every later candidate with that N. `run_nmpc_scenario` returns a `diverged` flag: a candidate whose pitch crosses the divergence limit, or whose state becomes non-finite, is always penalised.

`trms/cache.py` — `signature()` and `ParameterCache`; see below.

`trms/plotting.py` — every figure, drawn with matplotlib's non-interactive `Agg` backend so the program runs on servers without a display. Colours are fixed per controller (blue for PID, orange for N-MPC) in every figure, and the reference is always a dashed grey line.

`trms/reporting.py` — turns telemetry into files: per-step and global metric tables, JSON, the compressed NPZ of N-MPC predictions, the Excel workbook, and `summary.md`.

The controller interface

The simulation never checks which controller it is running. Any object with three methods works:

Method	Purpose
`reset(x0, n_steps)`	clear internal memory (PID integrator and filter; N-MPC warm start) and pre-allocate logs for `n_steps` samples

Method	Purpose
`step(t, x, ref_now, ref_fn)`	return the input u and a dictionary of internal values to log
`describe()`	return the controller's settings for the output JSON

This "duck-typed" interface makes adding a third controller, such as a gain-scheduled PID, a matter of writing one new class.

How the external libraries do the mathematics

Library	Used for	Where
NumPy	vectorised telemetry, PID arithmetic, metrics	everywhere
CasADi	symbolic model, automatic differentiation, NLP construction, compiled plant simulation (`Plant` , `Function.map`)	`model.py` , `nmpc.py`
IPOPT (bundled with CasADi)	solving the NLP at each sample	`ca.nlpsol('nmpc', 'ipopt', …)`
SciPy	differential evolution (tuning), Brent's method (trim)	`tuning.py` , `model.py`
pandas	telemetry tables, CSV and Excel writing, pivots	`simulation.py` , `reporting.py`
matplotlib	all figures	`plotting.py`
openpyxl	Excel engine behind pandas	`reporting.py`
PyYAML	reading settings	`main.py`
hashlib, json, tempfile	cache signatures, files and atomic writes	`cache.py`

CasADi works in two phases. **Construction** happens once: symbols such as `ca.SX.sym('X', 6, 39)` stand for unknown numbers; arithmetic on them builds an expression graph of the whole NLP; and `nlpsol` derives the exact gradient, Jacobian and

Hessian graphs by automatic differentiation. With `expand: True` these graphs are flattened into fast scalar code. **Solving** happens 1,200 times per run: the compiled solver receives numbers, and IPOPT iterates using the derivative functions CasADi built. No derivative is ever approximated numerically.

The parameter cache

Tuning takes minutes to tens of minutes, so its results are saved and reused. The difficulty is knowing when a saved result is still valid. Reusing PID gains tuned for a different model or sample time would silently invalidate the comparison.

The cache solves this with a **signature**: a SHA-256 hash of everything that can change a tuning result.

```
\text{sig} = \operatorname{SHA256}\big(\operatorname{JSON}_{\text{sorted}}\
{\text{algo\_version},\ \text{model params},\ \text{simulation},\
\text{objective},\ \text{tuning scenario},\ \text{controller section}\}\big)
```

The controller section includes the search bounds and DE budget. JSON keys are sorted so that equal settings always produce the same text and therefore the same hash. Changing even one digit anywhere in the inputs produces a completely different signature.

On each run, for each controller:

1. compute the current signature;
2. if `tuned_parameters.json` holds an entry with that signature and `--retune` did not name this controller, load it;
3. otherwise tune, then write the new entry, replacing the old one.

Runs with `--quick` use a separate file, `config/tuned_parameters_quick.json`, so a smoke test never overwrites the full-budget results. The JSON file's structure is described in Part 3.

The cache file is written atomically: the new JSON goes to a temporary file in the same folder, which then replaces the cache in a single rename. An interrupted run (Ctrl-C, crash, full disk) can therefore never leave a half-written file that would silently discard every tuned result.

Numerical safeguards and performance (2026-10-14 review)

A code review on 2026-10-14 fixed four logic bugs and made the pipeline 1.4–30× faster per stage without changing the architecture, the libraries or the output files. On the cached parameters the PID telemetry was bit-identical before the anti-windup fix, and the N-MPC trajectory moved by at most 1.1×10^{-4} rad.

Bug	Effect	Fix
Non-finite tuning score replaced by 0 without penalty	a NaN candidate could win the optimisation	non-finite J or a `diverged` flag always adds the 1,000 penalty
PID integrator froze when one step crossed the limit	output could stay short of saturation, or at 0 with a large Ki	integrator advances exactly to the limit, then clamps (Part 1.2)
N-MPC tuner stopped before storing the out-of-limit sample	a diverging candidate escaped the penalty	the candidate is flagged as diverged
Metrics on a run that stopped early	reporting crashed and all outputs were lost	short windows report NaN, `settled = False`

Smaller fixes:

- the comparison time series no longer truncates both runs to the shorter one;
- saturation is measured against the real actuator limits;
- every model divisor is validated;
- the rest angle uses `atan2`;
- the N-MPC multipliers are shifted with the primal guess (Part 1.3).

There are no explicit matrix inversions anywhere in the code. The only factorisations happen inside IPOPT (MUMPS), which regularises a near-singular KKT matrix itself, and all 1,200 solves of the reference run succeed.

Stage (same 2-core machine and settings)	Before	After
PID evaluation run, 1,200 samples with telemetry	5.0 s	0.1–0.2 s
N-MPC evaluation run, 1,200 IPOPT solves	20.2 s	13.7 s

Stage (same 2-core machine and settings)	Before	After
`python main.py`, cached parameters	40.5 s	28.0 s
PID auto-tuning	519 s	142 s
N-MPC auto-tuning	2,039 s	1,181 s

The gains come from three changes:

- the compiled CasADi plant (about 70× faster per sample);
- pre-allocated, vectorised telemetry instead of per-sample dictionaries;
- N-MPC solver reuse plus consistently shifted multipliers.

Three further ideas were measured and rejected:

- JIT-compiling the NLP to C took 162 s to compile per horizon for only a 25 % faster solve;
- ordering the variables stage by stage gave no gain;
- CasADi's FATROP solver did not converge on this problem.

The remaining per-sample N-MPC time is IPOPT's own work. The regression tests in `tests/test_core.py` check each fix and the bit-identical plant.

Part 3 — Data dictionary and metric definitions

Every run writes 16 files in the PID folder, 18 in the N-MPC folder and 12 comparison files at the run's root. All of them derive from one per-sample telemetry table per controller: 1,200 rows (60 s at 50 ms) by 61 columns for the PID and 63 for the N-MPC.

Output files

File	Location	Contents
`telemetry.csv`	`PID/`, `N-MPC/`	one row per sample, every logged variable (dictionary below)
`step_metrics.csv`	`PID/`, `N-MPC/`	one row per reference step: overshoot, rise/settling time, errors, coupling
`global_metrics.csv` / `.json`	`PID/`, `N-MPC/`	44 whole-run metrics for that controller

File	Location	Contents
`controller_parameters.json`	`PID/`, `N-MPC/`	the controller's settings plus its full tuning report
`nmpc_predictions.npz`	`N-MPC/`	every predicted trajectory (below)
`README.txt`	`PID/`, `N-MPC/`	a short description of the folder
`00_…png` to `10_…png`	`PID/`, `N-MPC/`	per-controller figures (Part 4)
`comparison_timeseries.csv`	root	both controllers on one time axis: angles, errors, inputs, cumulative IAE and $\int u^2$, compute time
`comparison_step_metrics.csv`	root	both step tables stacked, with a `controller` and `step_id` column
`comparison_global_metrics.csv`	root	the 44 global metrics, one column per controller
`comparison.xlsx`	root	sheets `global_metrics`, `step_side_by_side`, `step_metrics_long`, `timeseries`, `telemetry_PID`, `telemetry_N-MPC`, `parameters`
`compare_01…06.png`	root	overlay and bar-chart comparisons (Part 4)
`summary.md`	root	human-readable result tables
`settings_used.yaml`	root	the exact settings of this run, for reproducibility

`telemetry.csv` column dictionary

Angles are in radians unless the name ends in `_deg`. Every row describes the state *at the start* of a sample and the input applied *during* it.

Columns	Meaning
`time`	sample time t_k [s]
`ref_pitch`, `ref_yaw` (+ `_deg`)	references r_k
`x_pitch`, `x_S_v`, `x_u_vv`, `x_yaw`, `x_S_h`, `x_u_hh`	the six states
`xdot_*`	the six state derivatives f(x, u)
`pitch`, `yaw` (+ `_deg`)	measured angles (copies of x1 and x4)
`err_pitch`, `err_yaw`	tracking errors e = r − y
`pitch_rate`, `yaw_rate`	true angular velocities Omega_v, Omega_h [rad/s]
`pitch_acc`, `yaw_acc`	dS_v/dt, dS_h/dt [rad/s²]
`omega_main_rad_s`, `omega_tail_rad_s` (+ `_rpm`)	rotor speeds
`thrust_main_N`, `thrust_tail_N`	rotor thrusts S_f·F [N]
`J_h`	pitch-dependent yaw inertia [kg·m²]
`torque_aero_pitch`, `torque_gravity_pitch`, `torque_centrifugal_pitch`, `torque_friction_pitch`	the four pitch torques [N·m]
`torque_aero_yaw`, `torque_friction_yaw`	the two yaw torques [N·m]
`u_pitch`, `u_yaw`	applied inputs, after saturation
`u_unsat_pitch`, `u_unsat_yaw`	the controller's requested input before clipping (PID: v_k; N-MPC: first planned input)
`du_pitch`, `du_yaw`	input change since the previous sample
`sat_pitch`, `sat_yaw`	true when the input is at its limit
`cum_IAE_pitch`, `cum_IAE_yaw`	running integral of abs(e)
`cum_int_u2_pitch`, `cum_int_u2_yaw`	running integral of u²

Columns	Meaning
`compute_time_ms`	wall-clock time the controller took this sample
`pid_P_*`, `pid_I_*`, `pid_D_*`	PID only: the three terms
`pid_integrator_frozen_*`	PID only: anti-windup active
`nmpc_solve_time_ms`, `nmpc_ipopt_iterations`	N-MPC only: solver effort
`nmpc_success`, `nmpc_return_status`	N-MPC only: did IPOPT converge, and its status text
`nmpc_cost`	N-MPC only: optimal J of this sample's problem
`nmpc_constraint_violation`	N-MPC only: largest dynamics-constraint residual
`nmpc_pred_pitch_next`, `_yaw_next`, `_pitch_end`, `_yaw_end`	N-MPC only: predicted angles one step ahead and at the horizon's end

`nmpc_predictions.npz`

A compressed NumPy archive (open it with `numpy.load`) holding the complete plan made at each of the 1,200 samples:

Key	Shape	Meaning
`t`	(1200,)	sample time of each plan
`X`	(1200, 39, 6)	predicted states x_0 … x_38
`U`	(1200, 38, 2)	planned inputs u_0 … u_37
`R`	(1200, 38, 2)	references the plan tracked

`config/tuned_parameters.json`

One entry per controller. The `signature` decides whether the entry is still valid (Part 2). An abridged real entry:

```
{
  "nmpc": {
    "signature": "6c38…",                    // SHA-256 of the tuning inputs
    "created_utc": "2026-10-13T11:39:23+00:00",
    "params": {
      "weights": {"q_pitch": 1.0, "q_yaw": 3.049, "q_rate_pitch": 0.00418,
                  "q_rate_yaw": 0.343, "r_pitch": 0.01276, "r_yaw": 0.3523},
      "horizon_steps": 38
    },
    "tuning_report": {
      "method": "scipy.optimize.differential_evolution (parallel, integer
horizon)",
      "objective": 2.3627, "n_function_evals": 224, "generations": 6,
      "wall_time_s": 2039.5, "convergence_history": [2.588, 2.478, …, 2.363],
      "bounds": {"q_yaw_log10": [-1.0, 1.5], …, "horizon_steps": [15, 40]}
    }
  },
  "pid": { "params": {"Kp_pitch": 0.456, "Ki_pitch": 0.396, …}, … }
}
```

(The `//` comments are explanatory only; JSON itself has no comments.) `generations` counts generations after the initial population, which is why it reads 6 for the N-MPC's 7 evaluated populations. `convergence_history` holds the best J after each generation and is plotted as `00_tuning_convergence.png`.

Metric definitions

Notation: $t_k = k \cdot T_s$, $T_s = 0.05$ s, $e_k = r_k - y_k$. Every integral is computed as a rectangle-rule sum, so $\int g\,dt \approx \Sigma\, g_k \cdot T_s$.

Integral Absolute Error (IAE) — `IAE_pitch`, `IAE_yaw`; unit rad·s.

$$\text{IAE} = \int_0^T |e(t)|\,dt \;\approx\; \sum_{k} |e_k|\, T_s$$

The total area between the response and the reference. It counts small and large errors in proportion to their size and is the most direct single measure of overall tracking quality. In aerospace terms, it measures how far, for how long, the vehicle was not where it was told to be.

Integral Squared Error (ISE) — `ISE_*`; rad²·s.

$$\text{ISE} = \int_0^T e(t)^2\,dt \;\approx\; \sum_k e_k^2\, T_s$$

Squaring punishes large errors far more than small ones, so ISE is dominated by the biggest excursions, such as the first moments after a step. It matters when a large deviation is dangerous, such as a structural or flight-envelope limit.

Integral of Time-weighted Absolute Error (ITAE) — `ITAE_*`; rad·s².

$$\text{ITAE} = \int_0^T t\,|e(t)|\,dt \;\approx\; \sum_k t_k\, |e_k|\, T_s$$

Late errors weigh more than early ones, so ITAE penalises slow settling and persistent oscillation. Three variants appear, differing only in where t is measured from: the whole-run metric uses time from the start of the run; the per-step metric uses time since that step; and the tuning cost uses time since the last change on either axis.

Root Mean Square Error (RMSE) — `RMSE_*`; rad.

$$\text{RMSE} = \sqrt{\frac{1}{K} \sum_{k=1}^{K} e_k^2}$$

A typical error size in the units of the angle itself, which makes it easy to interpret: the N-MPC's pitch RMSE of 0.087 rad means about 5° on a typical sample. It weighs large errors more heavily, like ISE. `max_abs_error_*` reports the single largest error.

Control effort — `effort_int_u2_*`, `effort_int_abs_u_*`; s.

$$E_2 = \int_0^T u(t)^2\,dt \approx \sum_k u_k^2\,T_s, \qquad E_1 = \int_0^T |u(t)|\,dt \approx \sum_k |u_k|\,T_s$$

For a motor driven by voltage u, electrical power scales roughly with u², so E_2 is a proxy for energy drawn. On an aircraft this is battery capacity or fuel. Here it is dominated by the large constant u_v needed to hold the beam against gravity, which both controllers must pay.

Input total variation (TV) — `total_variation_u_*`; dimensionless.

$$\text{TV}(u) = \sum_{k=1}^{K-1} |u_k - u_{k-1}|$$

The total distance the input travels up and down: how much the actuator is worked, independent of its average level. High TV means rapid reversals, which cause motor heating, gear and bearing wear, current spikes and, on aircraft, fatigue of actuator linkages and structure. Related columns are `max_abs_du_*` (the largest single jump), `mean_u_*` and `rms_u_*`.

Saturation fraction — `saturation_fraction_*`: the share of samples within 0.05 % of the actuator range of either limit: u ⩾ u_max − 0.0005 (u_max − u_min), or the same distance above u_min. For the ±1 limits used here this means u ⩾ 0.999 or u ⩽ −0.999. Measuring against the real limits keeps the metric correct for non-unit or asymmetric actuators. Time at a limit is time without any reserve authority to reject an unexpected gust or disturbance, so certification practice typically requires margin.

Compute time — `compute_time_mean_ms`, `_median_ms`, `_p95_ms`, `_max_ms`, and `realtime_budget_exceeded_fraction`, the share of samples whose computation took longer than Ts = 50 ms. On a real flight computer, a late control update is a missed deadline. The worst case (max, p95) matters more than the average.

Step-response metrics

Each reference step j changes the reference on one axis from r_from to r_to, a step of size Δr = r_to − r_from. Its window starts at the step and ends at the next reference change on either axis. Define the sign of the step and the normalised progress:

$$\sigma = \operatorname{sign}(\Delta r), \qquad p(t) = \frac{y(t) - r_{\text{from}}}{\Delta r} \quad (0 = \text{start},\ 1 = \text{target})$$

Overshoot — `overshoot_pct`. How far the response passes beyond the target, as a percentage of the step:

$$\text{OS} = 100 \cdot \frac{\max\!\big(0,\; \max_{t \in \text{window}} \sigma\,(y(t) - r_{\text{to}})\big)}{|\Delta r|} \ [\%]$$

In flight, overshoot in attitude means the vehicle swings past the commanded orientation. That can violate an attitude limit, upset a payload, or demand a second correction. `peak_value` is the extreme value reached. `undershoot_pct` is the same measure in the wrong direction, against r_from: how far the response first moves away from the target.

Rise time (10–90 %) — `rise_time_10_90_s`:

$$t_r = t\big(p \ge 0.9\big) - t\big(p \ge 0.1\big)$$

using the first time each level is crossed. It measures speed of response. `delay_time_50_s` is the time from the step until the response first covers half the step.

Settling time (2 %) — `settling_time_2pct_s`. The time after which the response stays within a band of ±2 % of the step size around the target:

```
t_s = \min\Big\{\, t \ge t_{\text{step}} \;:\; |y(\tau) - r_{\text{to}}| \le
0.02\,|\Delta r| \;\;\forall\, \tau \in [t,\, t_{\text{end}}) \Big\} -
t_{\text{step}}
```

If the response is still outside the band at the end of the window, the step is marked "not settled" (empty or NaN in the CSV, `settled = False`). A window with fewer than two finite samples, for example after a run stopped early, reports every step metric as NaN and is marked not settled. This is the most practical single quality number: a pilot or autopilot can trust the manoeuvre as complete only once it has settled. A 2 % band is strict. For a 0.15 rad step it is ±0.003 rad (±0.17°).

Steady-state error — `steady_state_error`: the mean of r_to − y over the last 1 s of the window. A persistent offset means the vehicle holds the wrong attitude.

Per-step IAE, ISE, ITAE — the integral errors restricted to the step's window, with ITAE time measured from the step.

Cross-coupling — `cross_coupling_peak_dev_rad` and `_deg`. The largest deviation of the *other* axis from its own (constant) reference during the window:

```
C_j = \max_{t \in \text{window}_j} \big| y_{\text{other}}(t) -
r_{\text{other}}(t) \big|
```

It directly measures how much a manoeuvre on one axis disturbs the other. Pilots call this kind of interaction an unwanted off-axis response; it raises workload and, in severe cases, compromises handling qualities.

Aggregated step metrics

`global_metrics` also summarises the step table per axis: `mean_overshoot_pct_*` and `max_overshoot_pct_*`; `mean_settling_time_s_*` and `max_settling_time_s_*`, computed over **settled steps only**; `steps_not_settled_*`; and `mean_rise_time_s_*`. Read mean settling time together with `steps_not_settled`. The PID's yaw mean of 6.95 s describes the one step that settled, not the two that did not.

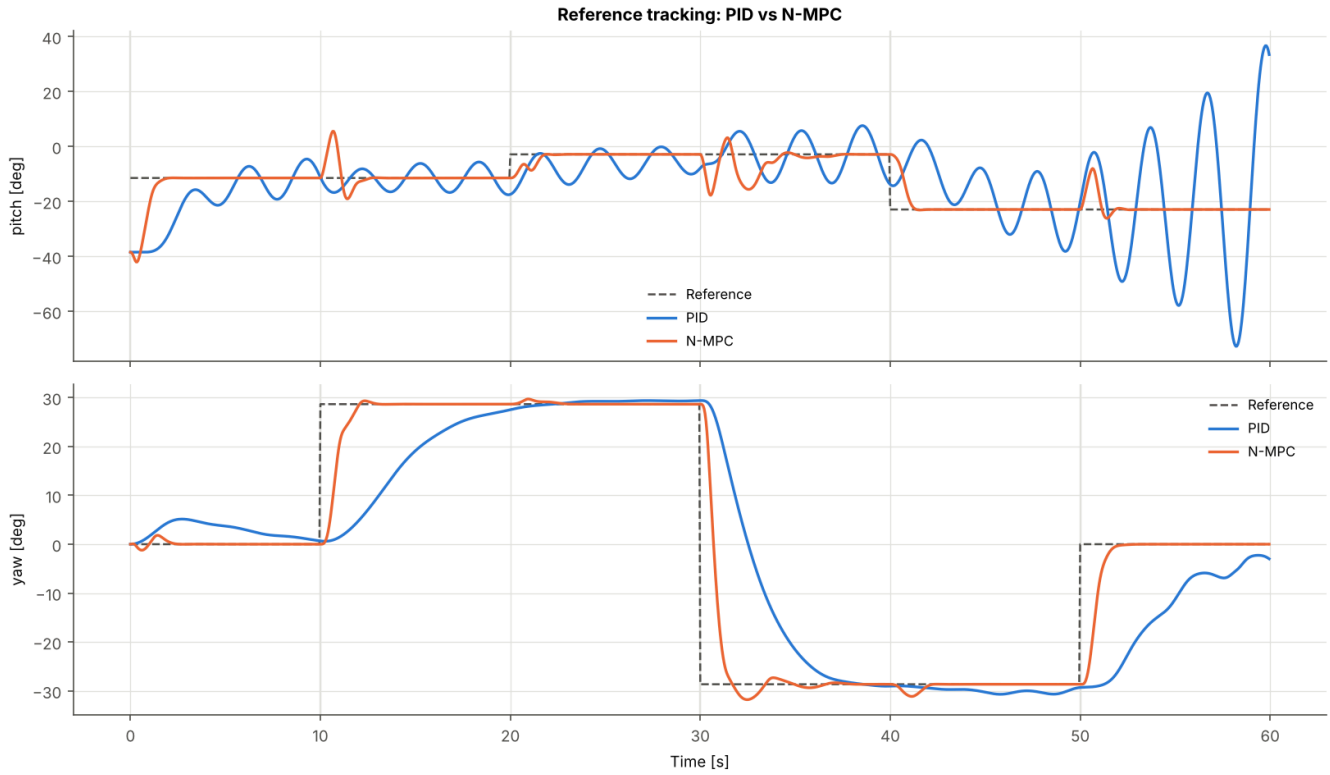
Part 4 — Visual analytics: how to read the graphs

Every comparison figure uses the same conventions: blue is the PID, orange is the N-MPC, a dashed grey line is the reference, and thin vertical grey lines mark reference changes. Time runs left to right over the 60 s evaluation. Angles are plotted in degrees, although the

code works in radians.

One principle helps with every graph. Before reading the lines, find the vertical marks and ask which axis changed reference there. A movement on the *same* axis is the controller's response to its command. A movement on the *other* axis is a cross-coupling disturbance.

Reference tracking (`compare_01_tracking.png`)



What it shows. The top panel is pitch, the bottom is yaw. Each panel overlays the reference and both controllers' actual angles.

How to read a step response. At each vertical mark on a panel's own axis, follow the coloured line from the old reference level to the new one.

- How steeply it climbs is the **rise time**.
- How far it passes the dashed line is the **overshoot**.
- How long until it flattens onto the dashed line is the **settling time**.
- Any gap left between line and reference at the end of the window is **steady-state error**.

At t = 0 s, the N-MPC (orange) lifts pitch from rest (−38.5°) to −11.5° in under 2 s and stays there. The PID (blue) takes about 3 s to arrive and then swings around the target with a period of about 3 s and an amplitude of roughly ±5°. In yaw at t = 10 s, the N-MPC turns to

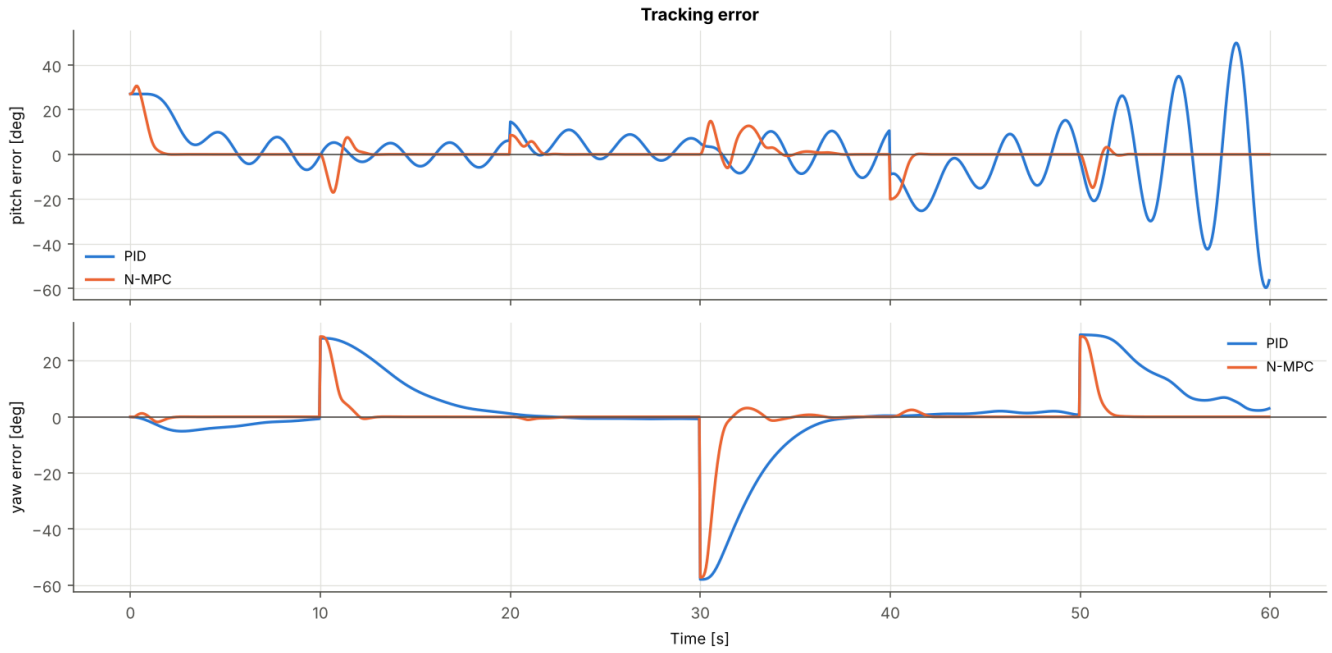
+28.6° in about 2.5 s; the PID needs about 10 s, approaching slowly without overshoot.

How to tell cross-coupling from overshoot. Overshoot is a bump on the *same* axis that was commanded, immediately after its own step, in the direction of the step. Cross-coupling is a bump on the *other* axis, whose reference did not change. Look at the pitch panel at t = 10, 30 and 50 s. These are yaw steps, yet the N-MPC's pitch jumps by up to 17° and recovers within about 2–3 s. Pitch received no new command there, so these bumps are coupling, not overshoot. Their physical cause is the tail rotor's reaction momentum (coupling path 1 in Part 1.1): the N-MPC drives the tail motor hard to turn fast, and the beam pitches in reaction.

The effect runs the other way too. At t = 0 the main rotor spins up to lift the beam, and its reaction momentum turns the beam in yaw. The PID's yaw drifts to about +5° before its yaw loop recovers. The N-MPC's yaw stays within about ±2°, because its model predicts this reaction and it fires the tail rotor to cancel it before a large error forms.

How to spot instability. A growing oscillation, where each swing is larger than the last, is the signature of an unstable loop. After the pitch step to −23° at t = 40 s, the PID's swings grow from about ±10° to beyond ±40° by t = 60 s. This happens at the operating point where the main rotor's thrust gain is highest, as Part 1.2 predicted.

Tracking error (`compare_02_error.png`)

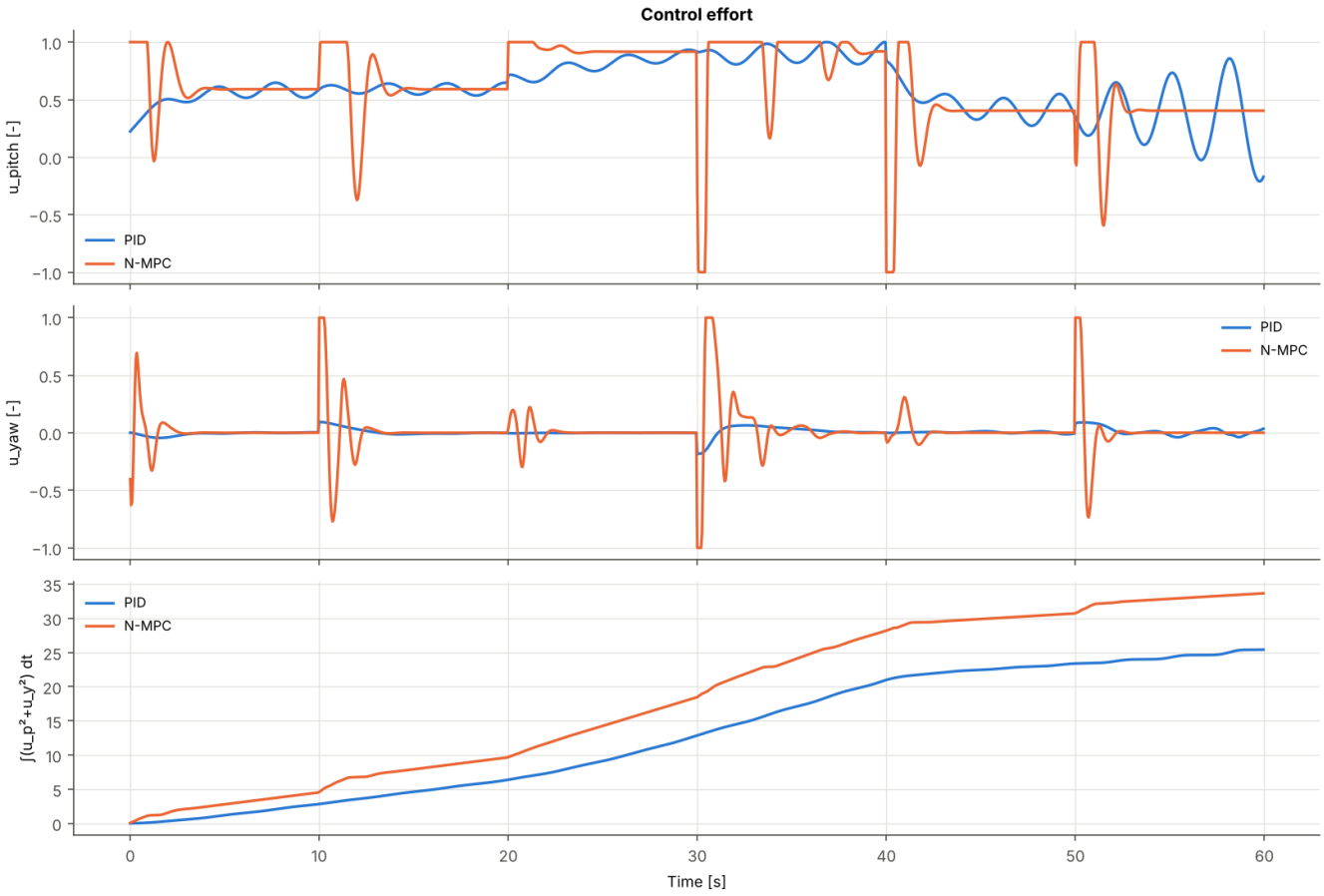


The same information as the tracking plot, re-centred so that the reference becomes the zero line: e(t) = r(t) − y(t). This makes small errors visible that would be lost against a 60° axis.

- A **spike at a step** that decays to zero is a normal transient.
- A **spike on the axis that was not stepped** is cross-coupling.
- An **oscillation that does not decay** is a limit cycle or, if growing, instability.
- A **constant offset** is steady-state error.

Note the sign. Positive error means the beam is *below* (pitch) or *short of* (yaw) the reference.

Control effort (`compare_03_control_effort.png`)



What it shows. The top two panels are the applied motor inputs u_pitch and u_yaw, limited to ±1. The bottom panel is the running energy proxy $\int(u_p^2 + u_y^2)\,dt$.

How to identify saturation. Saturation appears as a line that runs flat along exactly +1 or −1. The signal is "clipped" at the actuator's limit. The N-MPC's pitch input does this repeatedly: at +1 for about 1 s after each upward pitch step, and at −1 for short bursts at t = 30 and 40 s. It is at a limit 18.8 % of the time. The PID is at a limit only 0.7 % of the time: two brief touches of +1 near t = 37 s and t = 40 s, while holding the beam near 0° needs almost full main-rotor input. Saturation on its own is not a fault. For the N-MPC it is

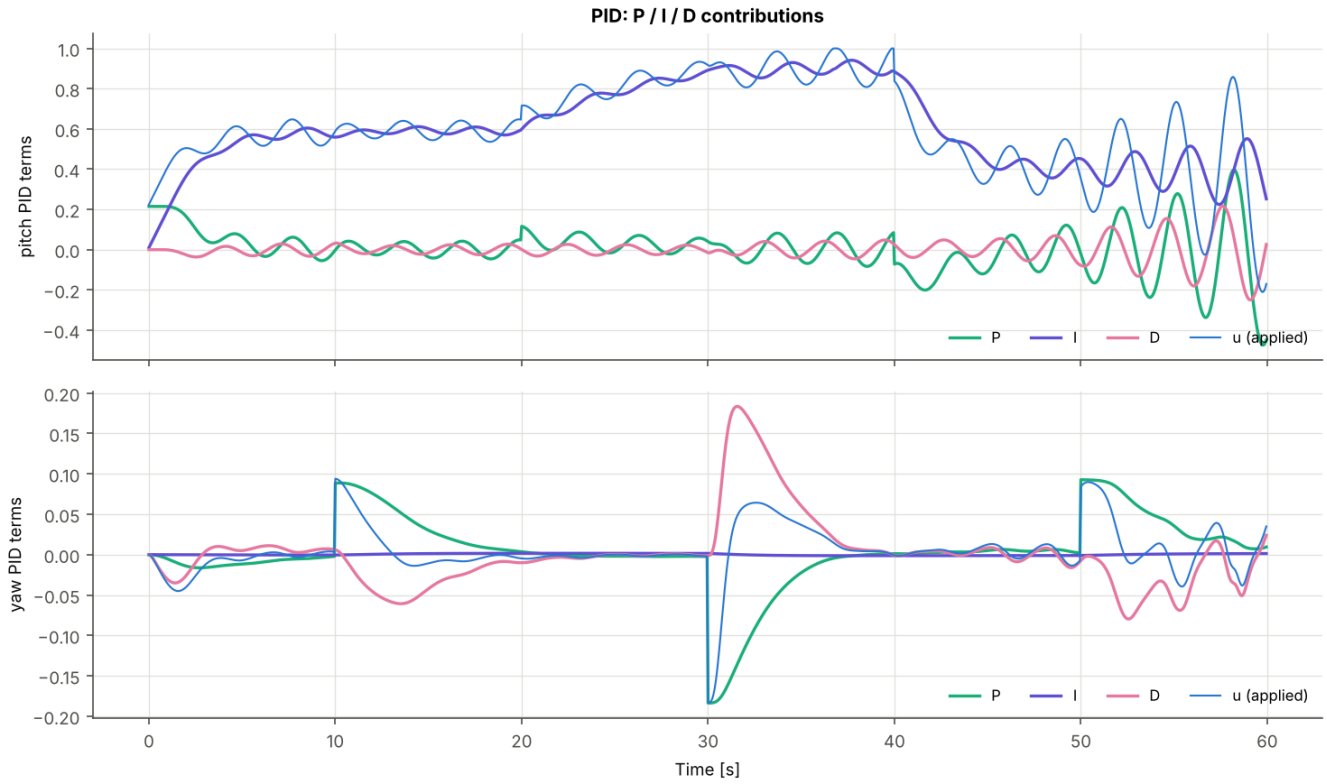
deliberate, because the limit is a constraint of the optimisation, and the optimum is often to use full authority briefly. For a PID, by contrast, saturation means the actuator cannot deliver what the controller demands, and the anti-windup must intervene.

Why the N-MPC commands sharp, precise pulses. The optimiser knows the main motor lags by 1.43 s. To move the beam quickly, the best plan is to over-drive the motor (input at +1) so its internal state rises fast, then cut back exactly when the predicted beam will arrive. The brief negative pulse is a brake that stops the beam at the target instead of letting it swing past. The tail input shows the same bang–counter-bang pattern. This is the "bang–bang-like" structure typical of time-optimal control: full effort in one direction, then the other, then hold.

Why the PID's smooth waves are ineffective. The PID's u_pitch oscillates gently between about 0.5 and 0.65 with the same 3 s period as the pitch error. The controller is chasing the swinging beam: each correction arrives through the slow motor after the swing has already reversed, so it partly feeds the oscillation instead of damping it. The waves look calm, but they never stop, and after t = 40 s they grow to cover most of the input range.

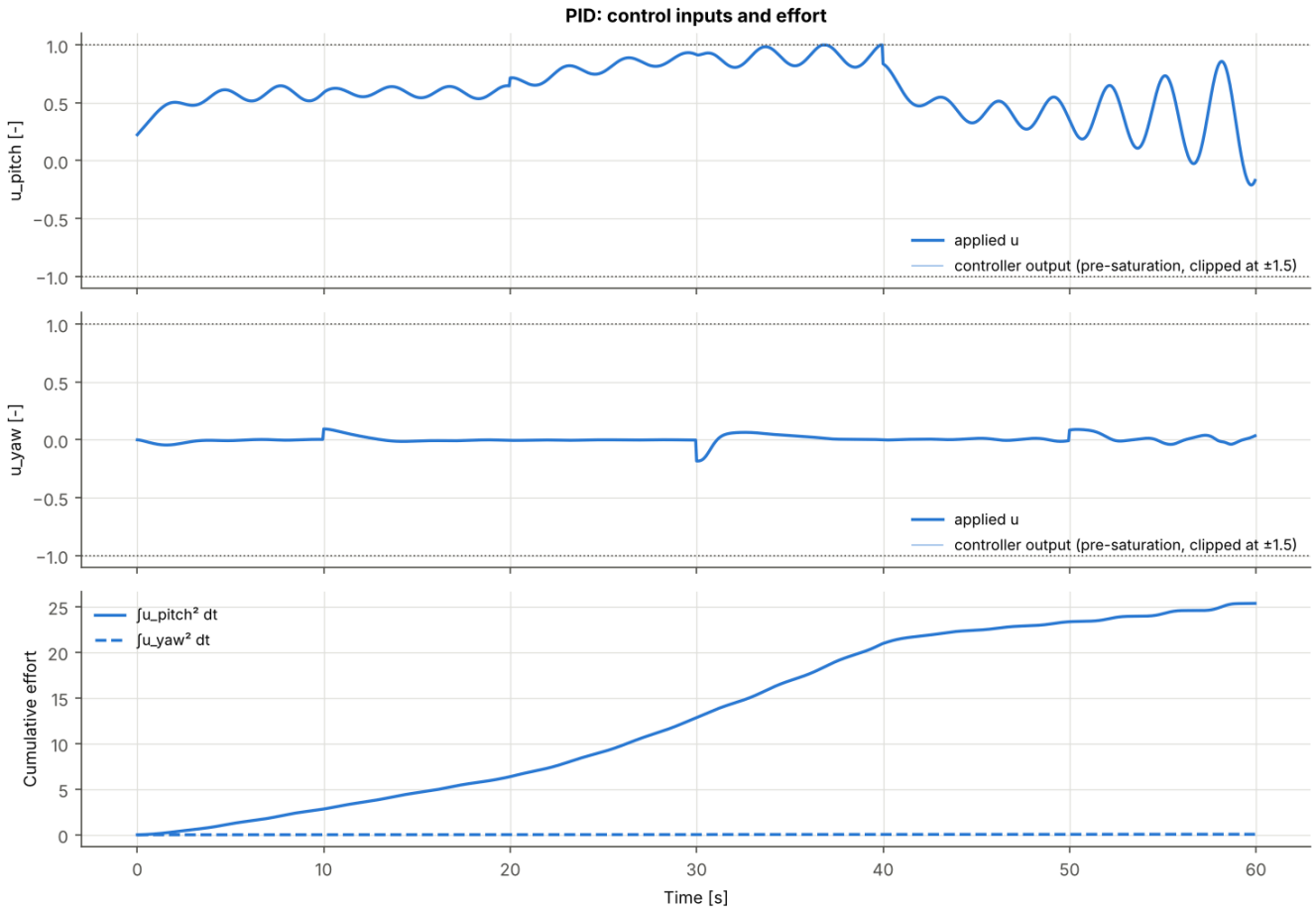
Reading the cumulative effort curve. Its slope is the instantaneous u^2, so a steep segment means heavy actuation and a gentle slope means light. Both curves rise steadily because holding the beam always needs u_v of about 0.4–0.9. The N-MPC's line climbs faster mostly in short steep jumps at each step, the cost of fast manoeuvres. The final values are 33.7 for the N-MPC and 25.4 for the PID.

Inside the PID (`PID/08_pid_terms.png`, `PID/03_control_inputs.png`)



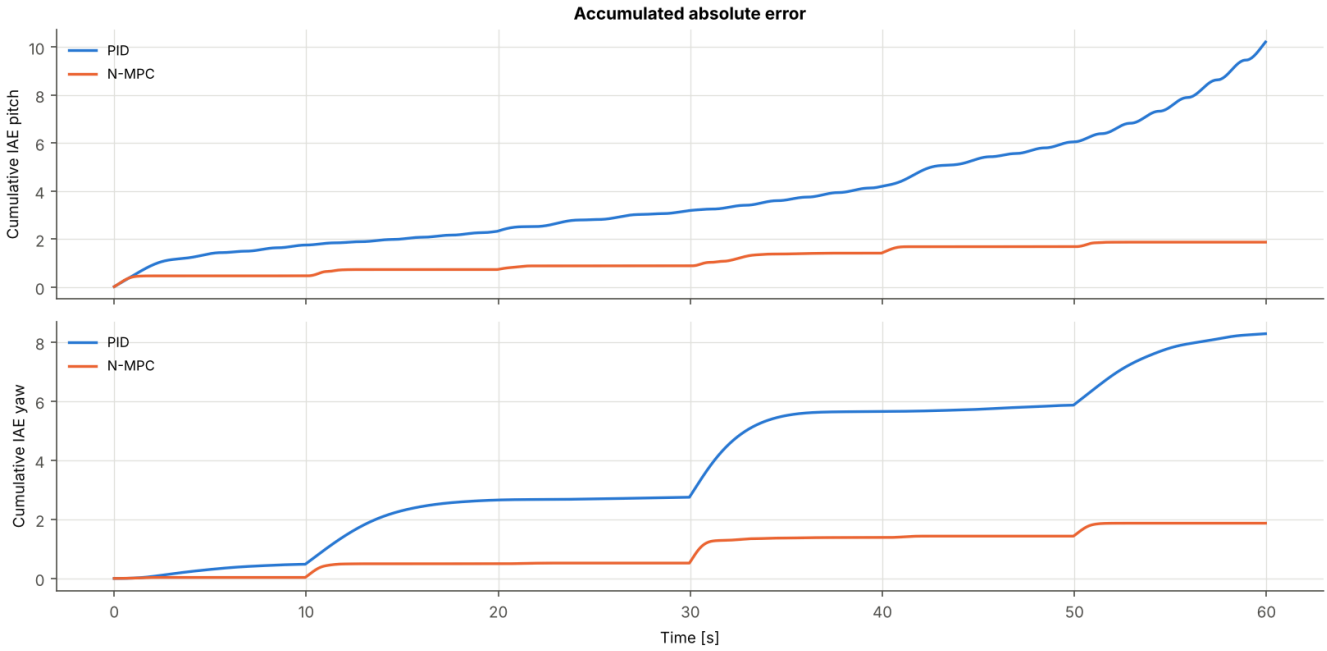
This plot splits the PID output into its three terms.

- **I (integral)** carries the large slow part: the 0.4–0.9 trim needed to hold the beam against gravity. Each pitch step shifts it to a new level.
- **P and D** carry the fast part and oscillate. D runs opposite to the motion, which is its damping role. Its amplitude is too small to stop the oscillation through the motor lag.
- The applied u is the sum, clipped to ±1.



In `03_control_inputs.png` the faint line is the *requested* input before saturation (v_k). Wherever the faint and solid lines separate, the actuator was clipped and anti-windup limited the integrator.

Cumulative IAE (`compare_04_cumulative_iae.png`)



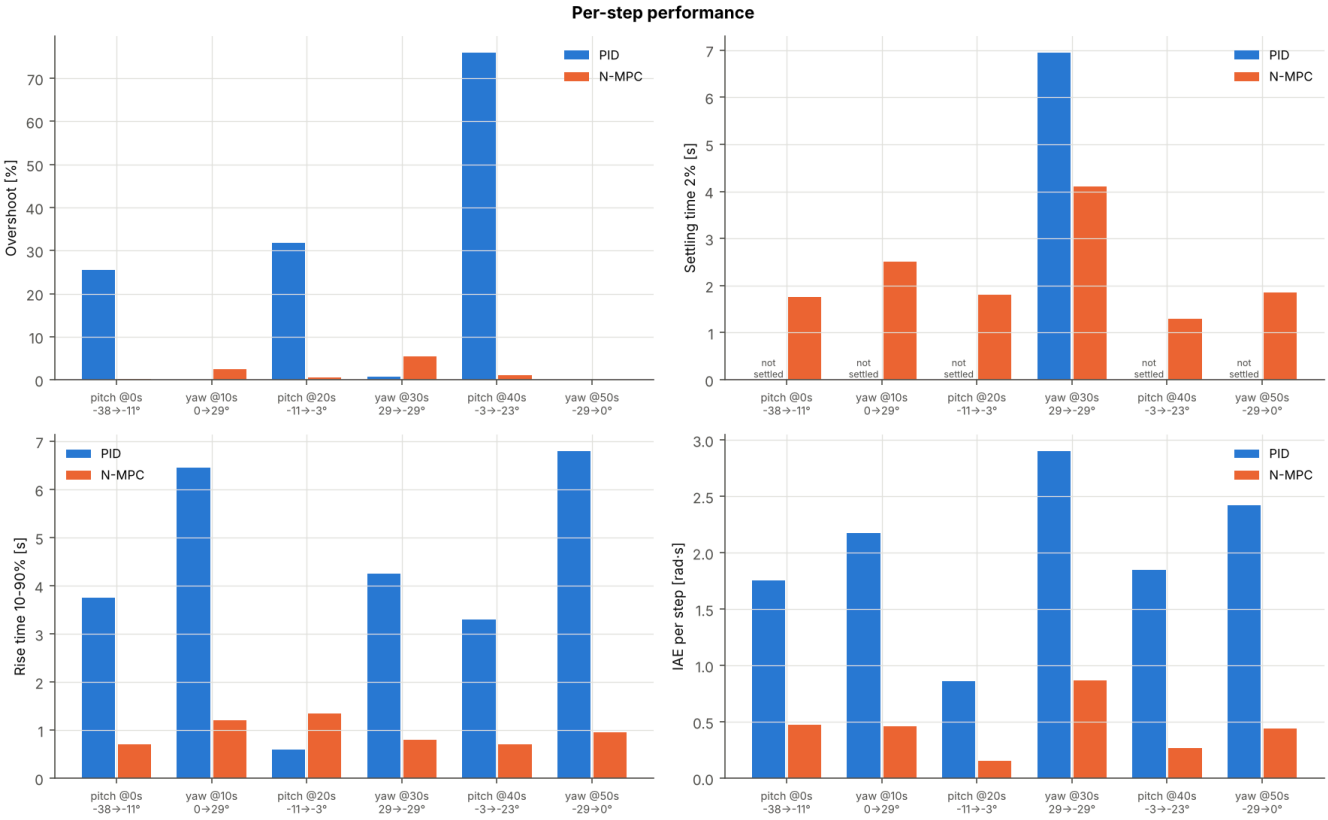
What it shows. The running integral $\int_0^t \text{abs}(e)\, d\tau$: the total error accumulated up to time t. The final value at 60 s is the whole-run IAE.

How to read it.

- The **slope** at any moment equals the current abs(e).
- A **flat** stretch means perfect tracking; a **steep** stretch means large error.
- A **staircase** (steep rises at each step, flat between) is healthy: errors only during transients.
- A **steady incline** that never flattens means a persistent error or an oscillation that never dies.
- A line that **gets steeper over time** means the error is growing, which signals instability.

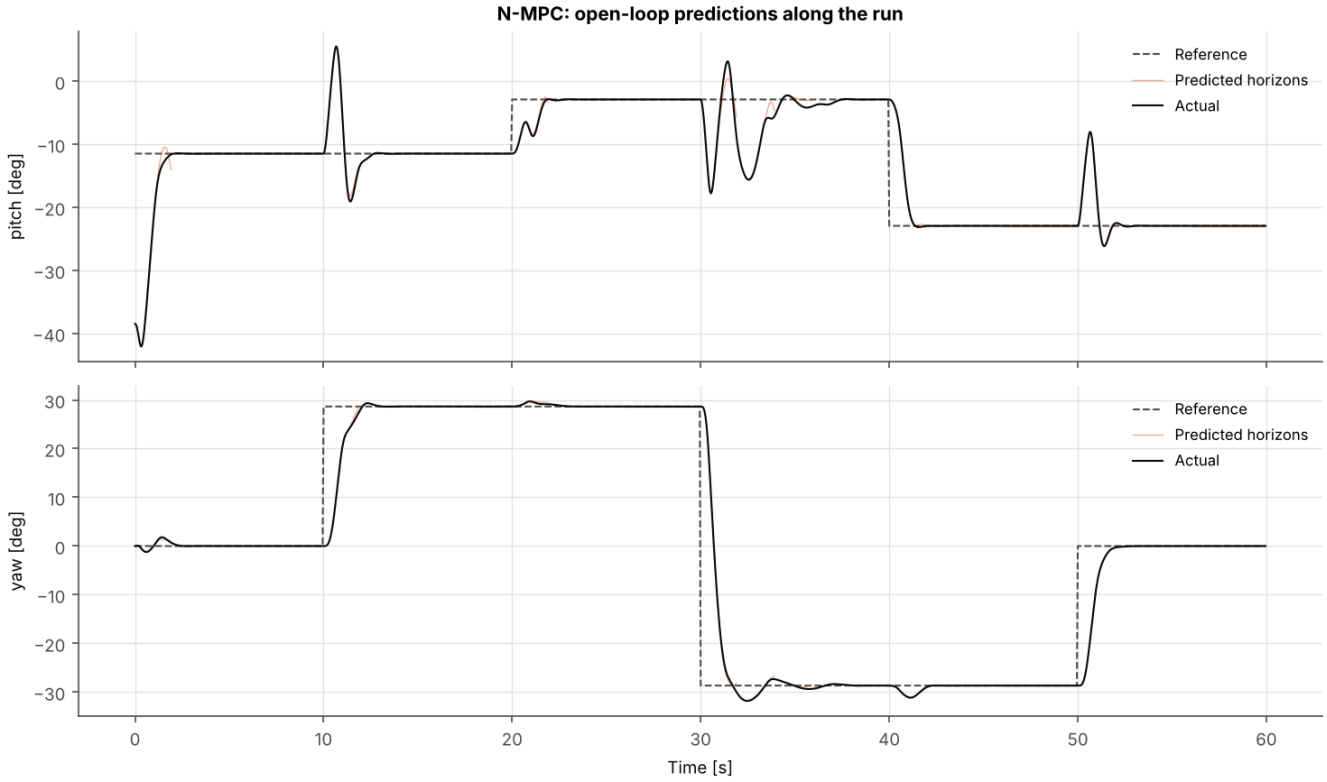
The N-MPC's pitch curve is a clean staircase ending near 1.9 rad·s. The PID's pitch curve never flattens because of the constant oscillation, and it steepens sharply after t = 40 s as the oscillation grows, ending near 10.2. The vertical gap between the curves at any time is the accumulated advantage of one controller over the other up to that moment.

Per-step bar charts (`compare_05_step_metrics.png`)



Four panels, each with one pair of bars per reference step (labelled with its axis, time and from→to angles). A bar marked "not settled" has no settling time, because the response never stayed inside the 2 % band. That outcome is worse than any finite bar, not better, even though it draws as a gap. Read the overshoot and settling panels together: a low overshoot with "not settled" (as for several PID yaw steps) means a response too slow to finish within the 10 s window.

N-MPC open-loop predictions (`N-MPC/09_nmpc_predictions.png`)



What the thin lines are. At every sample, the N-MPC computes a complete predicted trajectory 1.9 s (38 samples) into the future. That is the motion it expects if it applies its planned inputs. The plot draws one in every 40 of those predictions (30 in total) as a thin orange line starting at the time it was made. The thick black line is what actually happened, and the dashed line is the reference.

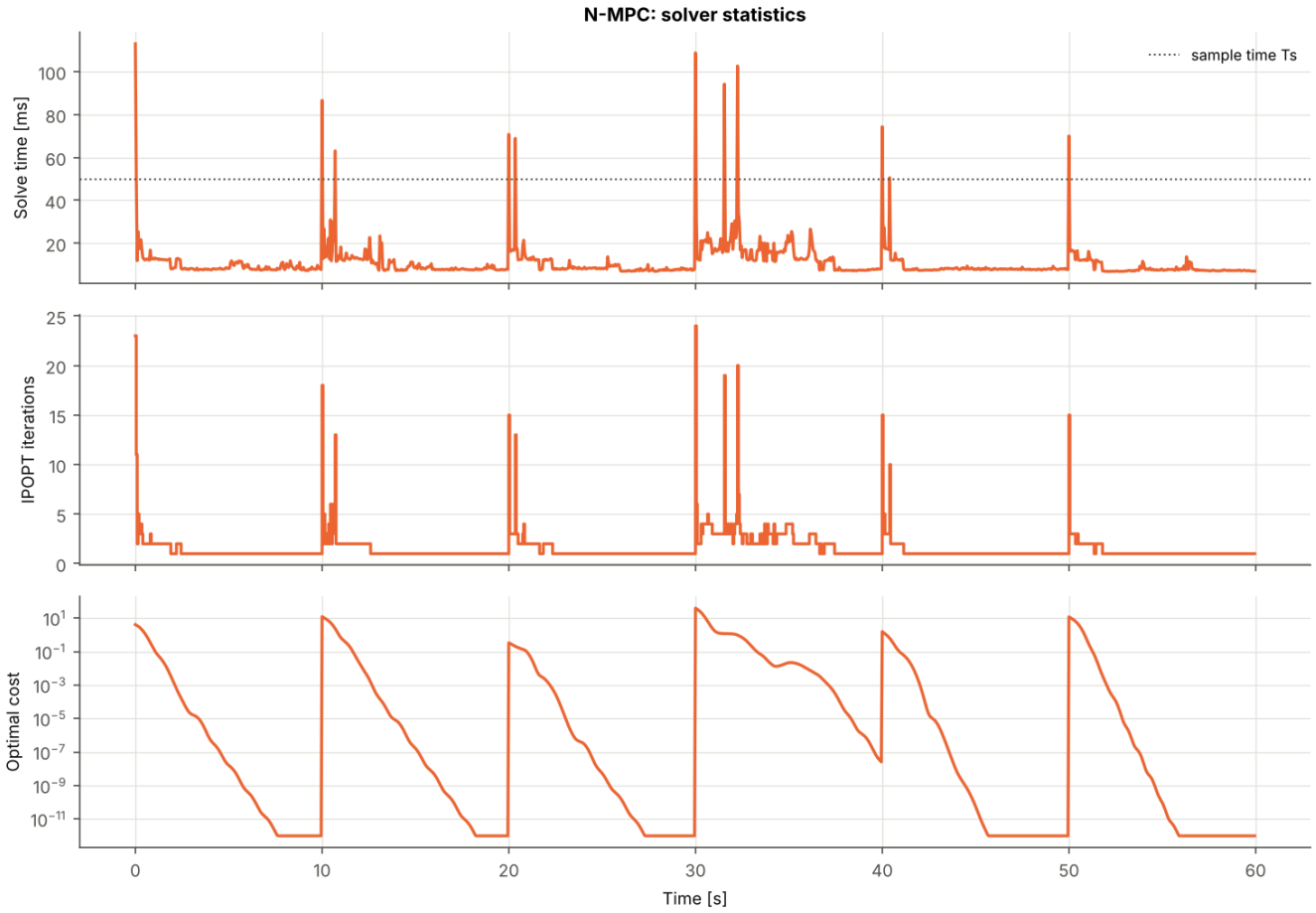
What agreement means. Almost everywhere, each thin line lies exactly on the black line. This means two things. First, the model is accurate: in this simulation it is the same equations as the plant. Second, the controller then carried out essentially the plan it had made, because nothing surprised it.

What disagreement means. Where a thin line peels away from the black line, something happened that the plan did not foresee. The clearest cases are the reference steps. A plan made at t = 9.9 s predicts holding still; at t = 10 s the yaw reference jumps, and every plan from then on is different. A visible example in the image is the early prediction near t = 1–2 s: it expected to approach the target from slightly above, and the next re-plan corrected the path.

What "seeing the future" really means here. The N-MPC sees the *consequences of its own actions* into the future. Within 1.9 s it knows that cutting thrust now means a fall later, that a tail-rotor burst will pitch the beam, and that the motor will lag. That foresight is why

it can brake before overshooting and cancel coupling before it grows. It does *not* see future reference changes, because preview is switched off. It learns about each step at the same moment as the PID. With `reference_preview: true`, the plans would bend toward an upcoming step before it arrives, and the N-MPC would start manoeuvring early.

N-MPC solver statistics (`N-MPC/08_nmpc_solver.png`)



Three panels show the price of that foresight.

- **Solve time per sample.** The dotted line is the 50 ms budget. Points above it are deadline misses (1.0 % of samples in the reference run). Wall-clock times depend on the computer and vary between runs: this figure's worst sample is about 113 ms at t = 0, while repeated runs ranged from about 110 to 310 ms. The IPOPT iteration counts and every trajectory are identical between runs.
- **IPOPT iterations.** Spikes cluster at reference steps, where the warm start from the previous plan is a poor guess and the solver must work harder.
- **Optimal cost on a log scale.** It jumps at each step (a large new error enters the cost) and decays by orders of magnitude as the beam settles.

Comparing these spikes with the tracking plot shows the computational pressure falls exactly when control matters most.

Part 5 — Results, limitations and glossary

In simulation, the N-MPC tracked about 5 times more accurately than the PID (total IAE 3.75 against 18.50 rad·s). It paid with 32 % more control effort, 4 times more input variation and about 200 times more computation per sample.

Whole-run results

These are the values in `docs/results/comparison_global_metrics.csv` (evaluation scenario, 60 s):

Metric	PID	N-MPC	Better
IAE pitch / yaw [rad·s]	10.21 / 8.29	1.88 / 1.87	N-MPC
ISE pitch / yaw [rad²·s]	3.68 / 3.63	0.45 / 0.92	N-MPC
ITAE pitch / yaw [rad·s²]	386 / 263	43.8 / 56.1	N-MPC
RMSE pitch / yaw [rad]	0.248 / 0.246	0.087 / 0.124	N-MPC
Max abs error pitch / yaw [rad]	1.04 / 1.01	0.53 / 1.00	N-MPC (pitch)
Steps settled (pitch / yaw)	0 of 3 / 1 of 3	3 of 3 / 3 of 3	N-MPC
Mean overshoot pitch / yaw [%]	44.5 / 0.2	0.6 / 2.6	N-MPC pitch, PID yaw
Mean rise time pitch / yaw [s]	2.55 / 5.83	0.92 / 0.98	N-MPC
Effort $\int u^2$ total	25.4	33.7	PID
Input total variation, total	11.0	44.6	PID
Largest single input jump pitch [–]	0.16	1.92	PID
Saturated samples, pitch	0.67 %	18.8 %	PID
Compute time mean / p95 [ms]	0.06 / 0.09	10.8 / 20.1	PID
Samples over the 50 ms budget	0 %	1.0 %	PID

The maximum yaw error is about 1.0 rad for both controllers. This is simply the size of the largest yaw step (+0.5 to −0.5 rad at t = 30 s), present at the instant the reference jumps, before any controller can act.

Per-step results

Step	PID OS [%]	PID settling [s]	N-MPC OS [%]	N-MPC settling [s]	N-MPC coupling on other axis [°]
Pitch −38.5° → −11.5° at 0 s	25.5	not settled	0.2	1.75	1.8
Yaw 0° → 28.6° at 10 s	0.0	not settled	2.4	2.50	17.0
Pitch −11.5° → −2.9° at 20 s	31.9	not settled	0.6	1.80	1.1
Yaw 28.6° → −28.6° at 30 s	0.7	6.95	5.5	4.10	14.9
Pitch −2.9° → −22.9° at 40 s	76.0	not settled	1.0	1.30	2.5
Yaw −28.6° → 0° at 50 s	0.0	not settled	0.0	1.85	14.9

Two patterns stand out.

- **Asymmetric coupling.** The N-MPC's pitch steps barely disturb yaw (1–2.5°). Its yaw steps disturb pitch by about 15–17°, because the tail rotor's reaction momentum acts directly on pitch rate. This is a property of the plant and of the tuned weights ($q_y \approx 3 \times q_p$), not a defect of the solver.
- **PID overshoot is inflated by the limit cycle.** The PID's pitch overshoot is measured on a beam that was already oscillating when the step arrived. For the same reason its `undershoot_pct` at t = 20 s reads 69 %: the beam happened to be at the bottom of a swing.

What the comparison does and does not show

It shows that, with a perfect model and full state measurement, a well-tuned N-MPC handles this plant's low damping, motor lag, varying gain and coupling far better than a well-tuned, fixed-gain, decentralised PID.

It does **not** show that N-MPC would win by the same margin on hardware. The main limitations are:

1. **Perfect model.** The prediction model and the simulated plant are the same equations. Real parameters differ from the paper's, and that mismatch hurts the model-based controller most.
2. **Full state, no noise.** The N-MPC reads all six states exactly. A real TRMS measures only the two angles, through encoders with quantisation, so an observer would be needed.
3. **Pure PID.** Gravity feed-forward and gain scheduling, the standard remedies for TRMS pitch, were deliberately excluded.
4. **Unverified errata.** The tail-thrust coefficient (0.0801 vs the paper's 0.8080) should be checked against the laboratory manual.
5. **Real-time margin.** IPOPT misses the 50 ms deadline on about 1–1.5 % of samples.
6. **Finite tuning budget.** Both searches stopped at their iteration limit; DE guarantees no global optimum.
7. **One scenario.** Results are for one 60 s step sequence. Sinusoidal tracking, disturbances and parameter variations would test robustness further.

Glossary

Term	Meaning
Anti-windup	logic stopping a PID integrator from growing while the actuator is saturated
Automatic differentiation	exact computer calculation of derivatives by applying the chain rule to the program's operations
CasADi	open-source library for symbolic modelling, automatic differentiation and optimisation
Cross-coupling	the effect of one input or axis on another axis
Damping ratio (zeta)	how quickly oscillations die out; 1 = no overshoot, near 0 = long ringing
Decentralised control	independent controllers per axis, ignoring interactions
Differential evolution	population-based, derivative-free global optimiser

Term	Meaning
Equilibrium / trim	a state, and the constant input, at which nothing changes
IPOPT	interior-point solver for large nonlinear programs
KKT conditions	the first-order optimality conditions of a constrained optimisation
Limit cycle	a sustained oscillation of fixed amplitude in a nonlinear loop
MIMO / SISO	multiple-input multiple-output / single-input single-output
Multiple shooting	posing an optimal-control problem with both states and inputs as variables, linked by dynamics constraints
NLP	nonlinear program: optimisation with nonlinear cost or constraints
Prediction horizon	how many future samples the MPC plans over (N = 38, 1.9 s)
Receding horizon	re-planning at every sample over a window that moves forward
RK4	fourth-order Runge–Kutta numerical integration
Saturation	an actuator held at its physical limit
Settling time	time until the response stays within a band (here ±2 %) of the target
Signature (cache)	hash identifying the exact inputs a cached result was computed from
Warm start	starting an optimiser from the previous solution
Zero-order hold	holding a digital input constant between samples

Sources

- arXiv:1306.4883, "Fault-Tolerant Control of a 2 DOF Helicopter (TRMS System) Based on H∞" — model structure and parameters.
- Project repository: github.com/aybarsavci/TRMS-Kontrol — code, cache and reference results.